



VRIJE
UNIVERSITEIT
BRUSSEL



Graduation thesis submitted in partial fulfilment of the requirements for the
degree of de Ingenieurswetenschappen: Computerwetenschappen

EXPLORING PLM ENSEMBLES

Analyzing How Protein Language Models
Can Be Combined to Improve
Downstream Predictive Performance

Ahmed Khalil-Allah Atmani

June 2026

Promotor: prof. dr. Wim Vranken

Advisor: Haochen Li

sciences and bioengineering sciences

Abstract

Protein Language Models (pLMs) have been shown to capture meaningful patterns from large protein sequence datasets such as UniRef, generating latent representations of proteins that can be used to train predictive models for downstream tasks in bioinformatics. In this thesis, we implemented and evaluated multiple methods for combining a selection of twelve different pLMs varying in specialization, size, base architecture, and training objective, and analysed their effects on predictive performance using the DRGN dataset, which was used as a binary classification task for cancer driver mutations. We experimented with feature-level and decision-level, as well as parallel and layered ensemble architectures, in addition to three distinct strategies for combining wild-type and variant sequences. Furthermore, we analysed which pLMs contributed most to the predictions, and which combinations yielded the best predictive performance. The results show that using exclusively the difference between the wild-type and variant representations leads to the best predictive performance, as it forces the task learners to focus on mutation-specific information while avoiding an increase in input dimensionality, unlike the other two variant combination strategies. Additionally, our experiments indicate that the predictive performance of pLM combinations depends strongly on their interaction: some embedding spaces complement each other, while others partially overlap or even dominate one another depending on the pairing. Overall, ESM-2, UniRep, and Ankh3 produced the most informative embeddings with respect to the downstream task considered in this study.



VRIJE
UNIVERSITEIT
BRUSSEL



Proefschrift ingediend met het oog op het behalen van de graad van Master of Science in de Ingenieurswetenschappen: Computerwetenschappen

ONDERZOEKEN VAN PLM ENSEMBLES

Analyse van combinatiemethoden voor eiwit-taalmodellen (Protein Language Models) om downstream predictieve performantie te verbeteren

Ahmed Khalil-Allah Atmani

Juni 2026

Promotor: prof. dr. Wim Vranken

Advisor: Haochen Li

wetenschappen en bio-ingenieurswetenschappen

Abstract

Protein Language Models (pLMs) hebben aangetoond dat ze betekenisvolle patronen kunnen leren uit grote proteïnesequentiedatasets zoals UniRef, waarbij ze latente representaties van proteïnen genereren die gebruikt kunnen worden om predictieve modellen te trainen voor downstream tasks in bio-informatica. In deze thesis hebben we meerdere methoden geïmplementeerd en geëvalueerd om een selectie van twaalf verschillende pLMs te combineren, die variëren in specialisatie, grootte, basisarchitectuur en trainingsdoel, en hebben we hun impact op de prestaties op de DRGN-dataset geanalyseerd, die werd gebruikt als een binaire classificatietask voor cancer driver mutaties. We hebben feature-level en decision-level, evenals parallelle en layered ensemble-architecturen onderzocht, naast drie verschillende strategieën om wild-type- en variantsequenties te combineren. Daarnaast hebben we geanalyseerd welke pLMs het meest bijdroegen aan de voorspellingen, en welke combinaties de beste voorspellende prestaties opleverden. De resultaten tonen aan dat het uitsluitend gebruiken van het verschil tussen de wild-type- en variantrepresentaties de beste voorspellende prestaties oplevert, omdat dit de task learners dwingt om zich te focussen op mutatiespecifieke informatie en tegelijkertijd een toename in de inputdimensionaliteit vermijdt, in tegenstelling tot de andere twee combinatiestrategieën. Daarnaast wijzen onze experimenten erop dat de voorspellende prestaties van pLM-combinaties sterk afhangen van hun interactie: sommige embedding spaces vullen elkaar aan, terwijl andere elkaar deels overlappen of zelfs domineren, afhankelijk van de combinatie. Over het algemeen leverden ESM-2, UniRep en Ankh3 de meest informatieve embeddings voor de downstream task die in deze studie werd onderzocht.

Contents

1	Introduction	7
1.1	Background	7
1.1.1	Biology Background	7
1.1.1.1	Protein Sequences	7
1.1.1.2	Protein Dynamics	8
1.1.1.3	Sequence Variants	8
1.1.2	Deep Learning Background	9
1.1.2.1	Multi-Layer Perceptrons (MLPs)	9
1.1.2.2	Integrated Gradients Feature Attribution	9
1.1.2.3	Auto-Encoder Architecture	10
1.1.2.4	Transformer Architecture	10
1.1.2.5	Foundation Models	11
1.1.3	Protein Language Models (pLMs)	11
1.2	Related Work	12
1.3	Problem Statement & Motivation	12
2	Methods	13
2.1	Dataset	13
2.2	Protein Sequence Cropping	14
2.3	Ensemble Architectures	15
2.3.1	MLP Task Learners & the NoPLM Model	16
2.3.2	Representation Fusion Ensemble Architecture	16
2.3.3	Task-Level Ensemble Architecture	18
2.3.4	Task-Level with Meta-Learner Ensemble Architecture	19
2.3.5	Gradient Boosting Ensemble Architecture	20
2.4	Selected pLMs	21
2.4.1	General-Purpose pLMs	22
2.4.2	Domain-Specialized pLMs	22
2.5	Pipelines	22
2.5.1	Embedding Pre-Computation Pipeline	23
2.5.2	Ensemble Learning Pipeline	24
2.5.3	Cross-Validation	26
2.5.4	Technical Implementation Considerations	26
2.6	Experiment Description	27
2.6.1	Experiment 1: Wild-Type & Variant Combination Strategies	27
2.6.2	Experiment 2: Individual pLM Performance	28
2.6.3	Experiment 3: Ensemble Architecture Performance	29

2.6.4	Experiment 4: Feature-Level pLM Combination Analysis	29
3	Results	31
3.1	Experiment 1: Wild-Type & Variant Combination Strategies	31
3.2	Experiment 2: Individual pLM Performance	33
3.3	Experiment 3: Ensemble Architecture Performance	36
3.4	Experiment 4: Feature-Level pLM Combination Analysis	37
4	Conclusion	49
5	Future Work	51
6	References	53

Chapter 1

Introduction

Protein Language Models (pLMs) have become a powerful tool in bioinformatics by capturing complex biological patterns from large sequence databases. These models generate high-dimensional latent representations that can be used to train predictive machine learning classifiers for downstream tasks. However, individual pLMs vary significantly in their parameters, training objectives, and architectural specializations, meaning that a single model may not capture all relevant biophysical properties. This thesis systematically addresses this limitation by exploring the design and implementation of pLM ensemble architectures. We evaluate multiple ensemble methods using a selection of twelve different pLMs to analyse how combining distinct latent spaces affects overall predictive power. The downstream task considered in this study is the binary classification of cancer driver mutations using the DRGN dataset [1].

Firstly, some biology and deep learning background will be introduced in Chapter 1, followed by the description of our methods and experiments in Chapter 2. After that, the results of those experiments will be presented and discussed in Chapter 3 and 4. Lastly, some possible extensions and future work will be discussed in Chapter 5.

1.1 Background

This background section is divided into three parts. The first part introduces the biological concepts required to understand this thesis, the second part presents the deep learning methods relevant to this work, and the third part discusses protein language models and ensemble learning.

1.1.1 Biology Background

1.1.1.1 Protein Sequences

Proteins are at the core of this thesis, making an understanding of them essential before discussing the technical implementation of this project. Proteins are biomolecules that play a role in many biological processes within living organisms. These roles include, but are not limited to, the catalysis of biochemical reactions, signal transduction, and molecular transport [2, 3].

Proteins consist of one or more chains of amino acids (referred to as “residues”); this form is known as the primary structure. Such amino acid chains naturally form α -helices and β -sheets

due to atomic interactions between amino acids, resulting in the secondary structure of a protein. The tertiary structure is characterized by the three-dimensional folding of these chains due to intermolecular forces. When several proteins in their tertiary structure interact and form a complex, the resulting structure is referred to as the quaternary structure.

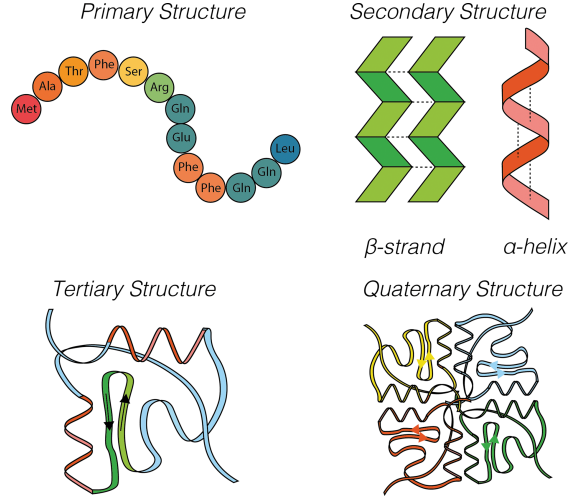


Figure 1.1: Primary, secondary, tertiary and quaternary structures of a protein. Image taken from [3].

The importance of the three-dimensional structure of proteins lies in the fact that a protein’s conformation directly determines its function [3]. Consequently, knowledge of protein structure is crucial in fields such as drug discovery and design.

1.1.1.2 Protein Dynamics

In the previous section, we established that the function of a protein is determined by its conformation. However, proteins continuously undergo microscopic motions and structural rearrangements due to thermal fluctuations [4]. This phenomenon is referred to as protein dynamics. Since protein function depends on conformation, these fluctuations can also result in changes in protein function [4]. The reason for the inclusion of this section is that some of the pLMs investigated in this work are specialized in the protein dynamics subdomain.

1.1.1.3 Sequence Variants

There are many types of protein sequence mutations, some of which result in a change or loss of protein function, while others are neutral [3]. In the dataset on which the ensemble models will be trained, only single-point substitutions are considered and labelled as driver mutations, i.e., mutations that contribute to the development of cancer, or passenger mutations, i.e., mutations that do not appear to contribute to the development of cancer. The original sequence before the mutation is called the wild-type, and the post-mutation sequence is called a variant.

1.1.2 Deep Learning Background

1.1.2.1 Multi-Layer Perceptrons (MLPs)

A multi-layer perceptron (MLP) is a term that refers to a simple neural network (NN) with more than one hidden layer [5, 6]. Mathematically, the output of an MLP can be described as a chain of function mappings:

$$f(\mathbf{x}) = \psi^{(L)}(\dots(\psi^{(2)}(\psi^{(1)}(\mathbf{x})))) \quad (1.1)$$

where $\psi^{(a)}$ represents the mapping of the a -th hidden layer, which is defined as:

$$\mathbf{h}^{(a)} = \text{ReLU}(\mathbf{W}_a \mathbf{h}^{(a-1)} + \mathbf{b}_a) \quad (1.2)$$

with $\mathbf{h}^{(0)} = \mathbf{x}$. Using this same notation, the final output of the MLP can be written as:

$$\hat{y} = \sigma(\mathbf{W}_L \mathbf{h}^{(L-1)} + \mathbf{b}_L) \quad (1.3)$$

where $\mathbf{W}_a$ and $\mathbf{W}_L$ are the trainable weight matrices of the a -th and final layer respectively, $\mathbf{b}_a$ and $\mathbf{b}_L$ are their corresponding bias vectors, $\text{ReLU}(z) = \max(0, z)$ is the activation function, and $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function [5].

The sigmoid activation function is used on the output of the last hidden layer (i.e., the output layer) because the downstream task considered in this thesis is a binary classification task, meaning that the model predicts a probability between 0 and 1. If the MLPs were instead used for a multiclass classification or regression task, the sigmoid function would typically be replaced by a softmax or identity activation function respectively [5].

An important addition to the MLP architectures is the use of dropout layers, which is a form of regularization. Their importance comes from the fact that pLM embeddings, similarly to the embeddings produced by other foundation models, are fixed representations in the sense that a given latent-space feature consistently captures the same underlying property or pattern for all samples [7, 8]. Without regularization, the MLP may become overly reliant on a subset of the very large amount of latent features generated by the pLMs, increasing the risk of overfitting. Dropout mitigates this issue by randomly disabling neurons with probability p during training, encouraging the model to process a broader set of features and improving generalization performance. In Chapter 2, the role of the MLPs in the learning process of the ensemble architectures is discussed.

1.1.2.2 Integrated Gradients Feature Attribution

Feature attribution is a method used to explain the decisions of machine learning models based on their predictions on a dataset, usually the test set [9]. Specifically, in the integrated gradients variant of feature attribution, importance scores are computed with respect to the change in gradients of the weights linked to the input layer of a neural network model. This reveals important and less important input features for a set of predictions, which will be useful in the analysis of how pLM latent spaces interact with one another in an ensemble architecture. More information concerning this analysis and the tools that were used will be given in Chapter 2.

1.1.2.3 Auto-Encoder Architecture

Some pLMs are pre-trained in a self-supervised manner using architectures inspired by the encoder-decoder paradigm. ProtT5 [7], being based on the T5 auto-encoder architecture [10], is an example. An auto-encoder is a deep learning architecture that consists of two distinct parts, namely an encoder and a decoder [5, 11]. During the pre-training phase, the encoder learns a latent feature space (the $\mathbf{y}$ vector in Figure 1.2) for the input, i.e., it learns to compress the input information in a way that still allows for its reconstruction [5]. At the same time, the decoder learns to reconstruct the original input (the $\mathbf{x}$ vector) from the latent representation generated by the encoder.

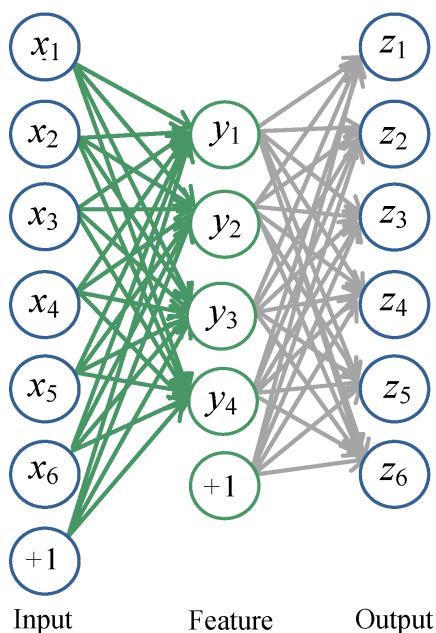


Figure 1.2: Diagram of an auto-encoder architecture, with $\mathbf{y}$ representing the latent vector. Image taken from [11].

Since this learning approach does not require explicit target labels, but instead compares the model's output to the original input, it is referred to as self-supervised learning rather than supervised or unsupervised learning [5]. One of the main reasons for the widespread adoption of self-supervised learning in the pLM literature is that very large sequence-only datasets such as UniRef50 can be used for training without the need for manual sample labelling. In other words, pLMs have access to sufficient training data to learn meaningful patterns from primary protein sequences that may contain information about protein function.

1.1.2.4 Transformer Architecture

Ever since the introduction of the attention mechanism by Google's research team [12], significant progress has been made in both natural language processing and bioinformatics, ultimately leading to the development of transformer-based models such as large language models (e.g.,

ChatGPT) and protein language models based on architectures such as BERT [13]. Attention, formally defined in Equation (1.4), is a mechanism that computes a weighted aggregation of value vectors based on similarity scores between query and key vectors. In other words, it allows a model to learn which parts of the input to attend to, depending on the input itself, in a single operation, in contrast to the more sequential attention approach used in recurrent neural networks (RNNs) [14, 15].

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1.4)$$

where Q refers to the query vector, K to the key vector and V to the value vector.

pLMs not based on the transformer architecture include UniRep [16], which is based on the sequentially attending mLSTM RNNs, and CARP [17], which is based on convolutional neural networks (CNNs), a class of models more commonly used in computer vision tasks.

1.1.2.5 Foundation Models

A foundation model is a large-scale model trained on a broad dataset that can subsequently be adapted to more specific tasks [18]. This allows the model’s knowledge of a domain to be leveraged when training on a more specialized dataset within the same domain, making it possible to obtain high-performance even with smaller specialization datasets. Training on the broad dataset is referred to as pre-training, while training on a more specialized dataset is called fine-tuning. In our pLM selection, some models are foundation models pre-trained on the broad UniRef dataset or a specialized dataset and are then frozen, while others are pre-trained on UniRef and subsequently fine-tuned on a more specialized dataset before being frozen.

1.1.3 Protein Language Models (pLMs)

Before the emergence of protein language models, other approaches based on thermodynamic simulations or multiple sequence alignment (MSA), such as AlphaFold, were used to capture patterns in protein sequences [19]. With the rise of natural language processing (NLP), similar techniques have been adapted to bioinformatics, where amino acid residues in protein sequences are treated the same way as words in sentences [20]. Before a protein sequence, which is typically represented as a string of residues (e.g., "MLKVA..."), is passed to a model, it must be tokenized, i.e., converted into a numerical representation. This tokenization varies across pLM families; for instance, ESM models also include start and end tokens.

Some protein language models are trained using a masked language modelling (MLM) objective, meaning that random residues in the input sequence are masked and must be predicted by the model, while others follow an autoregressive objective similar to natural language models, in which the model learns to predict the next (i.e., the last) token/residue in a sequence. The latter type of models are referred to as autoregressive models.

1.2 Related Work

Using ensembles of pLMs is not entirely new; ESM-1v is a pLM that is actually part of an ensemble of five identical models that were trained with different seeds for initialization of the weights to be used to learn downstream tasks by averaging the five outputs [21]. More about this model and how it was used along with the other pLMs in our thesis will be explained in Chapter 2.

Protein language models have been used to gain greater predictive performance on downstream tasks for some time, with D2Deep [22] and Venus-MAXWELL [23] being two of such examples. D2Deep combines pLMs with evolutionary data for better predictive performance on the same DRGN cancer driver mutation dataset that was used in this thesis. Similarly, Venus-MAXWELL combines pLMs with protein stability data to gain more predictive performance on a dataset in that subdomain.

More recent work has been published that explicitly make pLM ensembles, such as VarDCL [24], by combining the pLM embeddings in a specific manner. However, these methods are quite new, being released while we were working on our project, which is why they will not be discussed or used as baselines in this thesis.

1.3 Problem Statement & Motivation

What inspired us to analyse ensemble architectures is the famous BellKor solution [25] that won the \$1.0M Netflix Prize for building a high-performing recommender system. The winning solution was not an ingenious mathematical breakthrough leading to a novel algorithm, but rather an ensemble model that combined the best previous approaches to the problem, leveraging differently biased models so that their errors could partially cancel each other out.

Since protein language models exist in many architectures, sizes, and training objectives, they are natural candidates to be combined in order to improve performance on downstream tasks in bioinformatics. Implementing pLM ensembles is one aspect of the problem, but understanding why their combination can lead to improved predictive performance is an additional motivation of this work. Gaining insight into how the latent spaces of different pLMs interact may be key to further advances in this field.

Note that the code for our implementation of the ensemble models, pipelines, and experiments is publicly available in our GitHub repository¹. Our code is structured to allow easy extensions, for example to new ensemble architectures, datasets, and related components.

¹https://github.com/Ahmed-Atmani/pLM_Ensembles

Chapter 2

Methods

In this section, the dataset, ensemble architectures, and experimental pipelines used in this work are described in detail.

2.1 Dataset

As mentioned in Chapter 1, the dataset used in this thesis is the DRGN dataset [1], in which each sample is composed of a protein identifier, a variant, and a target label. In other words, each sample contains the identifier of a protein sequence that can be retrieved from the UniProt database¹, the variant describing a single-residue point-substitution mutation, and the target label, which is either 0 (negative) or 1 (positive).

The variant is represented in the following format:

`<original_residue><mutation_index><new_residue>`

where the residues are denoted using the standard one-letter amino acid codes. For example, the variant `S46N` indicates that the residue at position 46, which corresponds to an `S` (Serine) residue in the original sequence, is replaced by an `N` (Asparagine) residue.

The dataset is composed of a training set (`DRGN.txt`) and a test set (`ALL.txt`), with the latter additionally partitioned into two separate files (`NEUT.txt` and `DEL.txt`) according to the type of mutation: neutral and deleterious, respectively. Neutral substitutions have no significant effect on protein function, whereas deleterious substitutions do affect protein function. Since the objective of this thesis is to develop an ensemble model that generalizes well across mutation types, evaluation was performed on the complete `ALL` test set rather than on the `NEUT` and `DEL` subsets separately.

Since the DRGN dataset provides protein identifiers rather than the actual protein sequences, the sequences first had to be retrieved from the UniProt database. This process introduced several challenges. Our initial approach consisted of using the UniProt API to construct a new dataset containing the corresponding protein sequences by iterating through all samples in the DRGN dataset. However, this process did not proceed as smoothly as anticipated. The main reason is that the sequences stored in UniProt are continuously updated and corrected as new

¹<https://www.uniprot.org/>

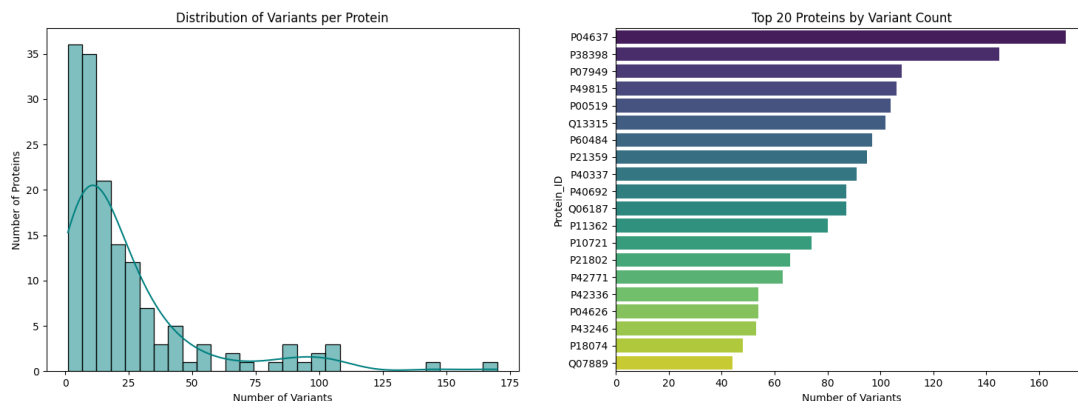


Figure 2.1: Overview of variants per protein in our pre-processed DRGN dataset.

experimental biochemical evidence becomes available. As a result, approximately 30% of the retrieved sequences contained variants for which the original residue specified by the dataset did not match the residue found at the corresponding mutation position in the retrieved sequence.

Furthermore, the original dataset paper [1] does not specify which version of UniProt was used, nor when the protein sequences were retrieved. Since the paper was published in 2021, several archived UniProt releases predating that year were evaluated. Among these, the November 2018 release yielded the lowest mismatch rate, reducing the proportion of variant residue mismatches down to approximately 10%. Consequently, this version was selected for all subsequent experiments. In some cases, protein identifiers present in the DRGN dataset could not be found in the selected UniProt release. These proteins, and therefore all samples associated with them, were removed from the dataset. An overview of the number of variants per protein sequence of the final dataset is shown in Figure 2.1.

After preprocessing and filtering, the final dataset contains 3644 training samples and 3644 test samples. Fortunately, this still provides a sufficient number of samples for the task under consideration. Both the training and test sets exhibit a class distribution of approximately 45% positive samples and 55% negative samples, which can be considered relatively balanced (see Figure 2.2). Nevertheless, evaluation metrics that account for class imbalance (e.g., F1-score) were also reported as a precautionary measure.

After this preprocessing, the datasets are saved as Python Pickle² files to be able to load the datasets with ease in our pipelines. The saved dataset contains the protein identifier, the sequence, the variant code and the target for each of the samples.

2.2 Protein Sequence Cropping

Some pLMs have a maximum input sequence length, which is often due to the limitations of the transformer architecture’s time complexity on which they are based. For instance, all pLMs from the ESM family have a limit of 1024 tokens, including the start and end tokens, reducing

²<https://docs.python.org/3/library/pickle.html>

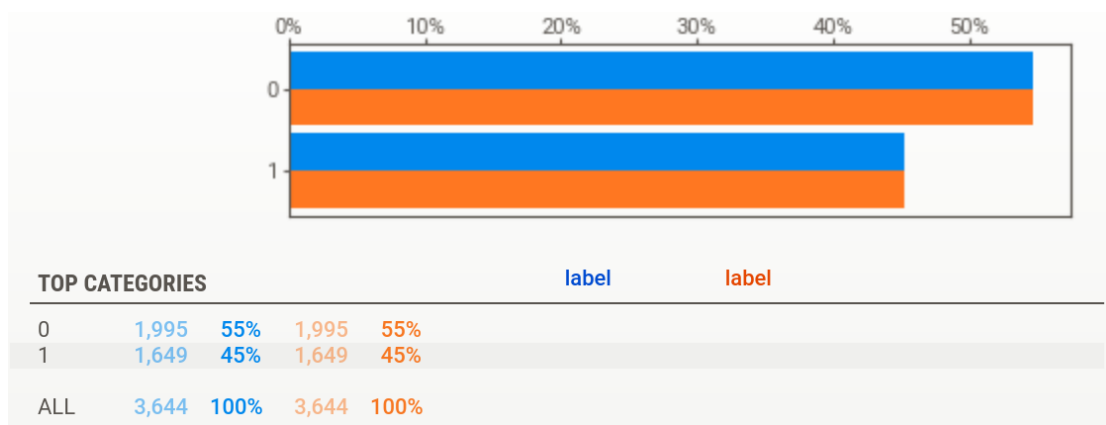


Figure 2.2: Visualization of the DRGN dataset’s balance.

the effective sequence length to 1022 residues. All other pLMs considered in this work can, in principle, process sequences of arbitrary length, except for AntiBERTy, which has an even lower limit of 512 tokens, or 510 residues excluding the start and end tokens. Therefore, all protein sequences were stored in three versions: the full sequence, the **1022-cropped** version (used by all pLMs except AntiBERTy), and the **510-cropped** version (used by AntiBERTy). The original intention was to use the same sequence length for all pLMs to minimize potential bias, hence the use of the 1022-residue sequences for all models. However, since AntiBERTy was added late in the project and supports a lower maximum sequence length, it was treated as an exception.

Since cropping residues inevitably results in a loss of information, one might expect this to reduce the amount of mutation-related information available to the models. However, studies such as [22] suggest that the sequence region surrounding the mutation site contains substantial predictive information, making this trade-off acceptable. To further minimize the impact of cropping, the cropping strategy itself is important. A naive approach, i.e., selecting the boundaries ($\text{mutation_index} \pm \frac{1022}{2}$), would always place the mutation site at the centre of the cropped sequence. This could introduce positional bias by encouraging the models to focus disproportionately on that particular position. To mitigate this effect, sequences were cropped such that the mutation index appeared at a random relative position within the cropped sequence while always remaining included in the resulting subsequence.

2.3 Ensemble Architectures

Ensemble architectures are architectures that combine multiple models together to form one aggregate model [26]. There are many different types of ensemble models, four of them being analysed in this thesis, each with its own advantages and disadvantages. However, depending on the research domain, the terminology and taxonomy may differ. For instance, [26] classifies ensemble models into two categories based on whether the sub-models learn the same task or not (parallel versus layered models), while [27] discusses alternative taxonomies, such as whether the sub-models learn from different latent spaces (deep representation-level ensembles) or not. To simplify the discussion of the ensemble models analysed in this thesis, we introduce two classifications:

- **Feature-level $\iff$ Decision-level**

The representation fusion ensemble architecture, which is the only feature-level architecture discussed in this thesis, combines the latent spaces of multiple pLMs before they are passed to a classifier. This provides a richer input representation for the task learner, which should, in theory, improve downstream predictive performance. The three remaining ensemble architectures discussed in this thesis belong to the decision-level category, as they combine the outputs of independent task learners. In theory, combining multiple independent predictions may reduce overall prediction error by allowing the errors of individual task learners to compensate for one another.

- **Parallel Models $\iff$ Layered Models**

In parallel ensemble architectures, all sub-models are trained simultaneously as part of a single training process, whereas in layered architectures, the sub-models are trained in multiple distinct phases. The representation fusion and task-level architectures are both considered parallel models, whereas the gradient boosting and task-level ensemble with meta-learner architectures are layered models.

The remainder of this section defines and describes the four ensemble architectures individually, as well as the multi-layer perceptron (MLP) configurations used within these architectures.

2.3.1 MLP Task Learners & the NoPLM Model

Multi-layer perceptrons (MLPs) are used throughout the four different ensemble architectures, since they are needed as they function as a bridge between the latent features from the pLMs and the prediction of the downstream task at hand. It is noteworthy that, in all four ensemble architectures in this document, those MLPs are the only parts that are actually being trained, as we are using pre-trained pLMs, which are essentially frozen foundation models. For a more in-depth explanation of how MLPs work, please refer to Chapter 1.

In addition to serving as the downstream task learner in the ensemble architectures implemented in this thesis, an MLP is also used as the basis of the “NoPLM” baseline model. Unlike the other architectures, the NoPLM model learns directly from tokenized protein sequences rather than from embeddings generated by a pLM, using an MLP. For this model, the tokenizer of the ESM-2.3B model was used, as it is one of the most promising pLMs in our selection. In other words, this model relies solely on sequence information and does not benefit from any additional protein knowledge encoded by protein language models. Including such a baseline not only allows us to assess whether pLM embeddings provide useful information beyond the raw sequences themselves, but also serves as a sanity check for the proposed ensemble architectures. Specifically, an ensemble model that consistently performs worse than the NoPLM baseline may indicate that the additional complexity introduced by the pLMs and ensemble mechanisms is not being utilized effectively.

For the exact values used for the hyperparameters of the MLPs that were used in this project, please refer to Table 2.2.

2.3.2 Representation Fusion Ensemble Architecture

The representation fusion ensemble is a feature-level parallel architecture in which the outputs of multiple foundation models are combined into a single fused embedding, which is subsequently used in the same way as a standard embedding vector for downstream learning tasks. In our

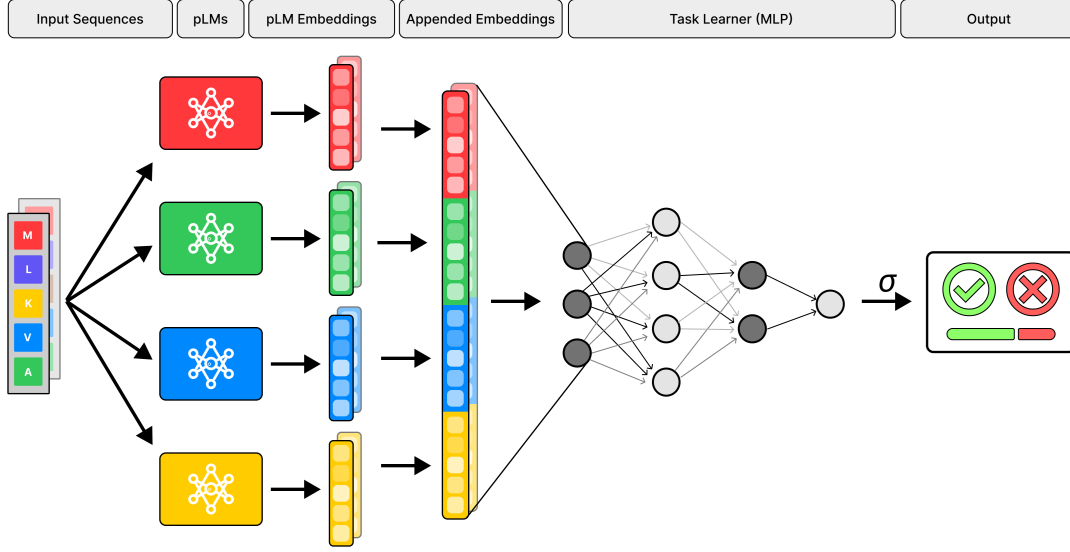


Figure 2.3: Diagram describing the representation fusion ensemble architecture.

particular case, this entails concatenating the embedding vectors produced by multiple pLMs into a single, high-dimensional embedding vector, as illustrated in Figure 2.3, or mathematically:

$$\text{new_emb}_{\text{append}} = [\text{emb}_1; \text{emb}_2; \dots; \text{emb}_P] \quad (2.1)$$

where P denotes the number of pLMs.

Rather than concatenating the embeddings, an alternative approach would be to combine them through averaging:

$$\text{new_emb}_{\text{average}} = \frac{\sum_{i=1}^P \text{emb}_i}{P} \quad (2.2)$$

However, since the pLMs considered in this work produce embedding vectors of different dimensionalities, direct averaging is not possible. Consequently, an additional transformation step would be required to project the embeddings into a common latent space before averaging them (e.g., AvgPooling). Such a transformation could alter the original latent representations and introduce additional sources of bias or information loss. For these reasons, we opted for simple concatenation, allowing the downstream task learner MLP to learn how to combine information from the different embedding spaces during training.

Having all embeddings combined into a single input representation allows the MLP to learn interactions between latent features originating from different pLMs, which may improve downstream predictive performance. On the other hand, the resulting high-dimensional input space increases the complexity of the learning problem, which may make optimization more difficult and increase the risk of overfitting.

2.3.3 Task-Level Ensemble Architecture

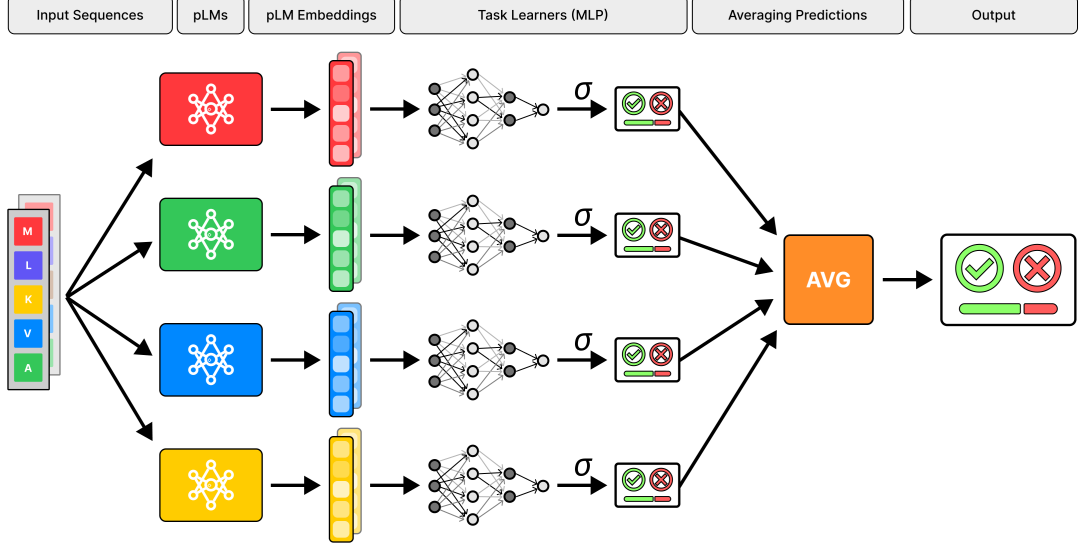


Figure 2.4: Diagram describing the task-level ensemble architecture.

The task-level ensemble architecture is a parallel decision-level ensemble architecture in which each foundation model has its own task learner that computes a prediction based on its corresponding pLM’s embedding space. The final output of the ensemble is then obtained by uniformly averaging the outputs of all task learners, as illustrated in Figure 2.4. Mathematically, the final output can be expressed as follows:

$$\text{ensemble_output}(x) = \frac{\sum_{i=1}^P \text{task_learner}_i(x)}{P} \quad (2.3)$$

where P denotes the number of pLMs and task_learner_i represents the MLP associated with the i -th pLM.

An alternative implementation would be to combine the predictions using majority voting, in which each task learner first produces a binary prediction and the final class is determined by the most common vote. Majority voting is one of the most commonly used output combination strategies in ensemble learning [26]. However, because the operation of rounding the predictions of the MLPs to 0 or 1 is not differentiable, gradients cannot be propagated through the voting operation during end-to-end training. As a result, the task learners cannot be optimized jointly through backpropagation, which is what the MLPs’ training is depending on. For this reason, we opted for the averaging method instead.

Since the individual task learners operate on different latent representations of the proteins, combining their predictions may improve robustness by reducing the variance associated with an individual learner. This principle is one of the main advantages of decision-level ensemble methods and is widely discussed in the ensemble learning literature [26].

2.3.4 Task-Level with Meta-Learner Ensemble Architecture

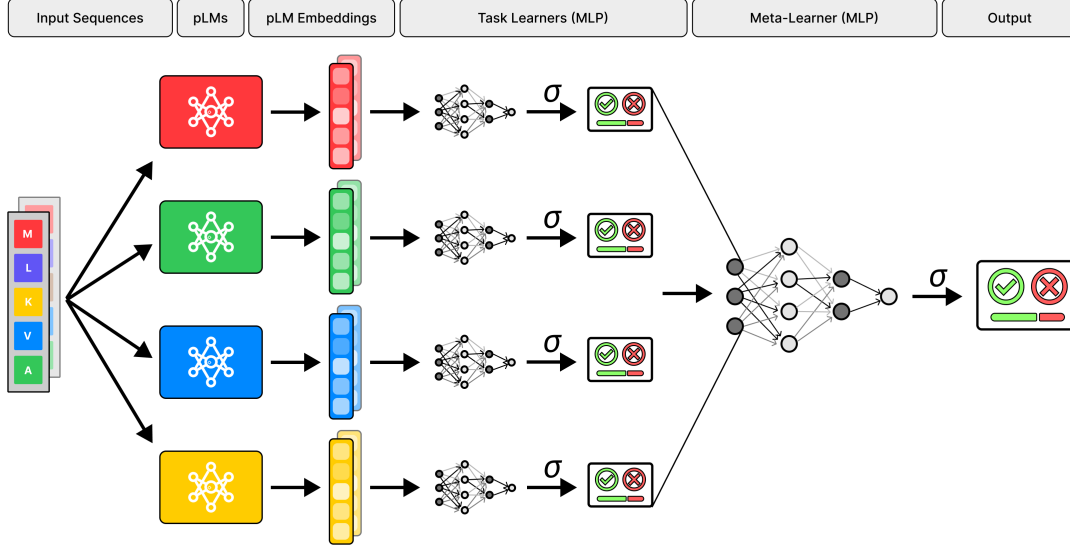


Figure 2.5: Diagram describing the task-level with meta-learner ensemble architecture.

This ensemble architecture is a variation of the task-level architecture discussed above that is layered instead of parallel. The main difference is the replacement of the averaging operation with an additional MLP, the meta-learner, which combines the outputs of the task learners in a learned manner (see Figure 2.5). Training this additional learner with the other MLPs simultaneously can be challenging since all MLPs initially have random weights. This means that the meta-learner would have to learn from the outputs of randomly initialized task learners, which could impede learning if done simultaneously. For this reason, we opted for the greedy/sequential version of this architecture instead of the joint version, which is why this architecture is classified as layered.

Sequential training of the model consists of two phases: first, the independent task learners are trained; second, the meta-learner is trained on their outputs. During the first phase, the simple average of the task learners’ outputs is used to compute the loss, meaning that this phase is equivalent to the task-level ensemble without a meta-learner. Once the first phase is completed, the task learners are frozen, and the meta-learner is introduced and trained in isolation, with the outputs of the frozen task learners.

Our choice of the sequential version of the architecture is based on empirical results, where we observed that the joint training variant fails to converge properly, likely for the reasons mentioned above. Here is the mathematical formulation of the task-level with meta-learner ensemble architecture:

$$\text{ensemble_output}(x) = M(\text{task_learner}_1(x), \text{task_learner}_2(x), \dots, \text{task_learner}_P(x)) \quad (2.4)$$

where P denotes the number of pLMs, task_learner_i represents the MLP associated with the i -th pLM, and M denotes the meta-learner.

This architecture, being a less constrained variant of the standard task-level ensemble, inherits its advantages while also being more expressive. This increase in expressiveness removes the need to predefine a combination strategy such as majority voting, since the meta-learner learns the optimal combination function directly from data.

The meta-learner MLP was chosen to be considerably less complex in our implementation compared to the task learners (one hidden layer of size 128 versus three hidden layers of size 1024 respectively), simply because an input size of P with a dimensionality of 1 is a very simple task. As Occam’s razor explains, the best hypothesis is the smallest hypothesis that fits the data, but not smaller [5].

2.3.5 Gradient Boosting Ensemble Architecture

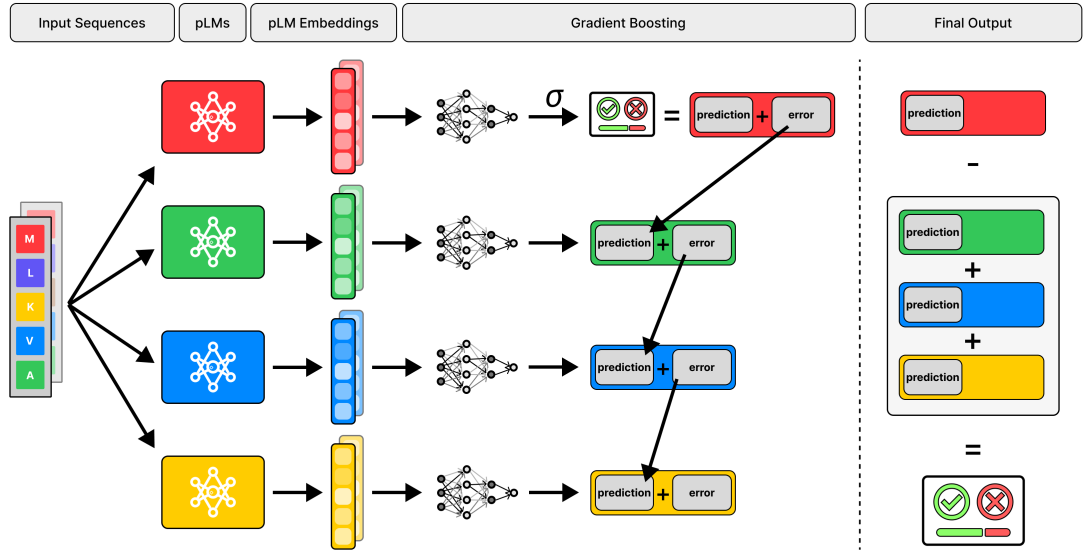


Figure 2.6: Diagram describing the gradient boosting ensemble architecture.

The gradient boosting ensemble architecture, originally proposed in [28] and later popularized by the XGBoost implementation [29] for decision-tree models, is a decision-level layered ensemble architecture. It operates by sequentially training sub-models, where each new sub-model attempts to learn the residual error of the previously trained models, while the first sub-model is trained directly on the downstream task. Unlike the other ensemble architectures discussed in this thesis, each training phase is associated with its own residual error term:

$$\text{error}_i(x) = \text{target}(x) - \sum_{k=1}^i \text{prediction}_k(x) \quad (2.5)$$

where i denotes the current training phase and $\text{prediction}_k(x)$ the output of the k -th sub-model. As can be seen on Equation (2.5), every subsequent sub-model, which corresponds to a pLM

MLP, attempts to correct the errors made by the preceding sub-models. The final prediction of the ensemble model is equal to the sum of the outputs of all sub-models.

Unfortunately, because this ensemble architecture was added relatively late in the project and due to time constraints, we were unable to get our implementation to work correctly. Our implementation exhibited progressively degrading downstream predictive performance as additional phases were trained, which suggest that an implementation error was present. Despite extensive debugging efforts, we were unable to identify the source of the issue, which ultimately led to the exclusion of this architecture from the experimental evaluation.

2.4 Selected pLMs

To have a more comprehensive analysis of pLM ensembles, we selected twelve pLMs that differ in base architecture, training data, amount of trainable parameters and embedding space dimensionality. An overview of the chosen pLMs can be found on Table 2.1. This section will briefly describe each of the twelve pLMs, which are grouped into two categories depending on its intended use/specialization.

Table 2.1: Architectural overview of the pre-trained protein Language Models (pLMs) utilized in the ensemble models.

pLM	Version Used	Year	Parameters	Emb. Dim.	Architecture Type	Specialization
<i>General-Purpose Models</i>						
UniRep	UniRep-mLSTM	2019	18M	1900	RNN (mLSTM)	General-Purpose
ESM-1b	ESM-1b-650M	2021	650M	1280	Transformer (BERT)	General-Purpose
ProtT5	ProtT5-XL-U50	2021	3B	1024	Transformer (T5)	General-Purpose
ESM-2	ESM2-15B	2022	15.0B	5120	Transformer (BERT)	General-Purpose
ProGen2	ProGen2-XL	2022	6.4B	4096	Transformer (Autoregressive)	General-Purpose
CARP	CARP-640M	2023	640M	1280	CNN	General-Purpose
ESM-C	ESM-C-600M	2024	600M	1152	Transformer (BERT)	General-Purpose
Ankh3	Ankh3-large	2025	1.1B	1536	Transformer (T5)	General-Purpose
<i>Domain Specialized</i>						
AntiBERTy	AntiBERTy	2021	26M	512	Transformer (BERT)	Antibody Specialized
ESM-1v	ESM-1v-650M	2021	650M	1280	Transformer (BERT)	Variant Specialized
SeqDance	SeqDance	2025	35M	45	Transformer (BERT)	Protein Dynamics
ESMDance	ESMDance	2025	35M	45	Transformer (BERT)	Protein Dynamics

Most of the pLMs are trained on the same UniProt database (among others for some pLMs) of protein sequences, meaning the differences in captured protein properties are mostly defined by their architectures and training methods.

2.4.1 General-Purpose pLMs

The ESM family of pLMs includes ESM-1b ([8]), ESM-1v ([21]), ESM-2 ([30]) and ESM-C ([31]³), listed in chronological order of release. All ESM models are based on the BERT architecture ([13]), which is an encoder-only transformer model. ESM-1v, being a specialized pLM, is described in more detail in the next subsection. Being based on the BERT architecture implies that these models are Masked Language Models (MLMs), meaning that during training random tokens are masked and predicted, as explained in Chapter 1. We initially intended to also include the state-of-the-art ESM-3 ([32]) model in our selection, but ultimately excluded it due to its substantially different architecture and API compared to the more standard sequence-only pLMs used in this thesis. To compensate for this exclusion, we included ESM-C instead, which is based on the ESM-3 lineage and was designed as a drop-in replacement for ESM-2.

ProtT5 ([7]) is also a well-established pLM based on the T5 architecture [10] rather than BERT. Another T5-based model in our selection is Ankh3, which aims to achieve comparable downstream performance with fewer trainable parameters, leveraging optimizations tailored to Google TPU-v4 hardware. ProGen2, in contrast, is based on an autoregressive GPT-style transformer that is not masked, i.e., it is trained to predict the next token given previous tokens. Finally, UniRep is based on recurrent neural networks (RNNs), in which the average of the hidden-state is returned as the embeddings, while CARP is based on convolutional neural networks (CNNs).

2.4.2 Domain-Specialized pLMs

ESM-1v, which was not discussed in detail in the previous section, is a variant-specialized model based on ESM-1b. It was fine-tuned on a variant dataset as part of an ensemble of five models with different random initializations to reduce prediction variance [21]. However, for consistency across models, we only consider the first of the five models in our experiments.

AntiBERTy ([33]), as the name suggests, is a BERT-based model pre-trained on antibody sequences instead of the usual UniRef databases. This model was included to assess whether antibody-specific pre-training leads to different learned representations of protein sequences.

SeqDance and ESMDance ([34]) are protein dynamics-specialized pLMs trained on the same dataset but with different initialization strategies. SeqDance is trained from randomly initialized weights, whereas ESMDance is initialized from frozen ESM-2 (35M parameters version), making them interesting comparison models for our experiments.

2.5 Pipelines

Most of the pLMs that are included in this project are fairly large, to the extent that they will not fit in the RAM of most off-the-shelf computer, let alone VRAM to accelerate the computation of the embeddings. Furthermore, loading all pLMs at the same time in memory makes this even more impossible, and is a challenge even for the VUB HPC⁴ supercomputer (with Nvidia H200 clusters) we have access to. Such is the rationale behind the splitting of the machine learning pipeline into two distinct parts; one pre-computing and storing the embeddings of all pLMs, and

³The preliminary print was cited because the research team behind the ESM models has not yet published a formal academic paper for ESM-C (<https://www.evolutionaryscale.ai/blog/esm-cambrian>)

⁴<https://hpc.vub.be/>

the other using those pre-computed embeddings to feed the task learners. This splitting of the pipeline has no effect on the embeddings that are generated, nor on the training of the MLPs, while making the computation more modular and accessible. The two pipelines will be discussed in the following subsections.

2.5.1 Embedding Pre-Computation Pipeline

As mentioned above, this first pipeline pre-computes and saves the embeddings of each pLM such that they can be used in the next pipeline. A more detailed overview of how the pipeline is implemented can be found in Algorithm 1.

Algorithm 1: Embedding pre-computation pipeline pseudocode.

```

1 # Load the train and test datasets
2 dataset = load_dataset()

3 foreach pLM do
4     # Load the model into (video) memory
5     plm_model = load_plm(pLM)

6     # Initialize an empty dictionary
7     embedding_dict = {}

8     foreach sample in dataset do
9         #Extract and save embeddings for each sample
10        embeddings = plm_model(sample.sequence)
11        mut_embeddings = plm_model(sample.mut_sequence)

12        new_id = sample.id + "_" + sample.variant
13        embedding_dict[new_id][0] = embeddings
14        embedding_dict[new_id][1] = mut_embeddings

15    # Persist the dictionary to disk
16    save_to_disk(embedding_dict)

```

As can be seen in Algorithm 1, the embeddings both the original sequence and the mutated sequence (i.e., the variant) are stored. To ease the fetching of the variant embeddings in the next pipeline, we have saved them using a new combined identifier of the form `<protein_id>_<variant>`. That way, the ensemble models have enough information to directly fetch the right variant embeddings of the dataset sample and immediately combine them to directly compute the output of the entire model.

Unlike the preprocessed dataset, the embeddings generated by each pLM were stored in separate HDF5 files using the `h5py`⁵ Python library rather than the `Pickle` module. The HDF5 file format was chosen because it supports partial loading of files, whereas `Pickle` files must be loaded entirely into memory before any part of their contents can be accessed. Since the embedding files are substantially larger than the entire cancer driver mutation dataset itself, as they contain large floating-point vectors for every sample, the ability to load only the required subsets (e.g., the training partition) improves the efficiency of the ensemble learning pipeline (see Section 2.5.2).

⁵<https://www.h5py.org/>

Before saving the embeddings of the sequences, the embeddings are normalized such that their values have a mean μ of 0 and a standard deviation σ of 1, i.e., the embeddings are standardized (see Equation (2.6)). This was done to mitigate the potential issue where one pLM’s embeddings would dominate another’s due to differences in the magnitude of latent feature values, which are determined by the design choices of the pLM rather than their relative importance.

$$\mathbf{x} = \frac{x - \mu}{\sigma} \quad (2.6)$$

Although most pLMs produce per-residue embeddings, meaning one embedding vector for each amino acid in a sequence, this work exclusively considers per-sequence embeddings. For pLMs that output per-residue embeddings, the per-sequence representation is obtained through average pooling across all residues. Using per-sequence embeddings allows to uniformly compare pLMs and simplifies both the implementation and analysis of the ensemble architectures. Furthermore, directly working with per-residue embeddings would substantially increase the computational and methodological complexity of the experiments.

It is worth noting that all embeddings were stored using 32-bit floating-point precision, except for ESM-2 and ProGen2, whose embeddings were stored using 16-bit half-precision floats. This choice was motivated by the substantial memory usage of the 15B-parameter ESM-2 model and the 6B-parameter ProGen2 model, which exceeded the practical memory constraints of both our consumer-grade hardware and VUB’s Hydra supercomputer clusters when operating entirely in 32-bit precision. Using half-precision floating-point representations substantially reduces memory consumption while having only a negligible impact on downstream predictive performance, according to [7].

2.5.2 Ensemble Learning Pipeline

All ensemble architectures, except for the task-level model with meta-learner, make use of the same typical batched machine learning pipeline. The exception arises because the task-level with meta-learner architecture, similarly to the discarded gradient boosting architecture, requires multiple training phases. Nevertheless, all training pipelines follow the same general machine learning workflow, consisting of loading a batch of data, computing the loss, and updating the model parameters using an optimizer step. In this section, the implementation and execution of each pipeline are described in detail. The values of all hyperparameters used throughout the experiments, which were selected empirically, are listed in Table 2.2.

For the training of the MLPs, we opted for the AdamW optimizer instead of conventional stochastic gradient descent (SGD). This choice was motivated by AdamW’s widespread adoption in the deep learning and ensemble learning literature, including works such as [12] and [23], as well as its proven effectiveness across a wide range of deep learning applications.

The representation fusion pipeline is relatively straightforward; training a representation fusion ensemble model simply consists of training the task learner that receives the fused embeddings as input. Each batch iteration entails fetching the protein identifiers and passing them to the ensemble model, which retrieves the corresponding embeddings from the aforementioned HDF5 files, concatenates them into a single fused embedding vector, and forwards it to the task learner

Table 2.2: Hyperparameter configuration used for the ensemble learning pipeline.

Hyperparameter	Value
Optimizer	
Optimizer	AdamW
Learning Rate	1.0×10^{-5}
Weight Decay	1.0×10^{-3}
Batch Size	64
Task Learner MLP	
#Hidden layers	3
Hidden size	1024
Dropout	0.3
Meta-Learner MLP	
#Hidden layers	1
Hidden size	32
Dropout	0.1
Early Stopping	
Early Stopping Window	5
Early Stopping Patience	20
Maximum Epochs / Runs	40
K-Fold Cross-Validation	
Cross-Validation Folds	5

MLP. The binary cross-entropy loss is then computed from the MLP’s output and backpropagated to update the model weights. It is worth noting that the aforementioned NoPLM model, as well as the regular task-level ensemble model, use the same training pipeline due to their similarly straightforward training procedures. These architectures differ in their internal structure, but are trained in the same manner.

For the task-level ensemble with meta-learner, the first training phase is identical to the pipeline described above. Once this phase is completed, i.e., when the task learner MLPs are done training, the task learner MLPs are frozen and the meta-learner MLP is activated. The second phase follows the same procedure as the first phase, except that only the meta-learner is updated through backpropagation while the task learners’ weights remain unchanged.

For the gradient boosting model, the same principle is used, except that the number of phases depends on the number of pLMs the model combines. After every training phase, the current pLM’s task-learner MLP is frozen, the next MLP is activated, and the next training runs begin. What makes this pipeline even more different than the others, is that the loss function also changes with the phases, since the previous MLPs’ predictions also contribute to the model’s final output.

Concerning the training loops, we implemented early stopping with a patience parameter. The reason for introducing patience is that the models needed several training epochs before achieving strong validation performance. This is because the models first need to learn meaningful basic patterns from the protein representations before they can attain higher validation scores. Without a patience mechanism, training would be terminated prematurely by the early stopping mechanism before the model has had the opportunity to improve.

2.5.3 Cross-Validation

Since the dataset is not perfectly balanced and contains multiple variants associated with the same wild-type protein sequence, it is important to reduce potential bias towards recurring sequences, which is why we employed stratified group K-fold cross-validation. K-fold cross-validation consists of splitting the training dataset into K folds and executing the training pipeline K times, each time selecting a different fold as the validation set. The performance measurements are then averaged across the K executions, ensuring that the validation scores are representative of the entire training set while reducing the variance of the results. The test set, however, must never be used in a way that influences training, as doing so would compromise the reliability of the test scores.

Using the grouped variant of K-fold cross-validation ensures that all variants associated with a given wild-type protein sequence are assigned to the same fold. As a result, the model is trained on one set of wild-type sequences and validated on a different set of previously unseen sequences, encouraging the MLPs to learn patterns related to the mutations rather than memorizing properties of the underlying wild-type proteins. Stratification ensures that the proportion of positive and negative samples remains as constant as possible across the training and validation folds, preventing situations in which a fold contains a disproportionately large number of samples from a single class, which could lead to overly optimistic or pessimistic validation scores.

It is important to note that the same fold-splitting seed is used within each experiment to ensure that identical data splits are used across different architectures and pLM combinations. It is also worth noting that only the ensemble models obtained from the first fold are saved, which will be used during the fourth experiment described later in this section.

2.5.4 Technical Implementation Considerations

This section summarizes several implementation challenges and design decisions that are not essential to the remainder of the thesis, but are relevant for the reproducibility of the experiments. As mentioned in Chapter 1, our implementation is publicly available on GitHub⁶.

- For pLMs that use additional tokens, such as start and end tokens, the corresponding per-residue embeddings are discarded before computing the mean per-sequence embedding.
- In the case of UniRep, the JAX-UniRep implementation⁷ was used because of its GPU acceleration through Google’s JAX library⁸, as well as its straightforward installation process, whereas the original implementation required the deployment of a Docker container.
- Some pLMs, such as CARP, did not support input batches by default and therefore had to be manually extended to support batching. Similarly, models such as ProGen2 did not provide automatic batch sequence padding and required additional implementation effort. Note that this does not affect the results of the experiments.
- ESM-C’s 6B-parameter variant was listed in the README documentation of its GitHub repository⁹, but was unavailable in the released code and therefore could not be loaded. As a result, we used the second-largest available variant containing 600M parameters.

⁶https://github.com/Ahmed-Atmani/pLM_Ensembles

⁷<https://github.com/ElArkk/jax-unirep>

⁸<https://docs.jax.dev/en/latest/>

⁹<https://github.com/Biohub/esm>

2.6 Experiment Description

This subsection will describe all the different experiments that were done in this thesis. The results of each experiment will then be presented and discussed in sections 3 and 4 respectively.

2.6.1 Experiment 1: Wild-Type & Variant Combination Strategies

For a model to be able to learn patterns within mutations, enough information, and, more importantly, the right information should be present in the input [5]. Ideally, the input space of an ML model should exclusively contain the features that are important with respect to the patterns that need to be learned. The reason is that having too much information in the input can mislead the model or even cause the model to learn “shortcut patterns” that do not reflect those the model was supposed to learn in the first place, while still scoring high on measures such as accuracy. An example of this phenomenon at work is the famous paper [35] in which the CNN model learned to classify wolves and dogs by learning to detect snowy backgrounds or grassy environments instead of actually looking at patterns in the animal itself.

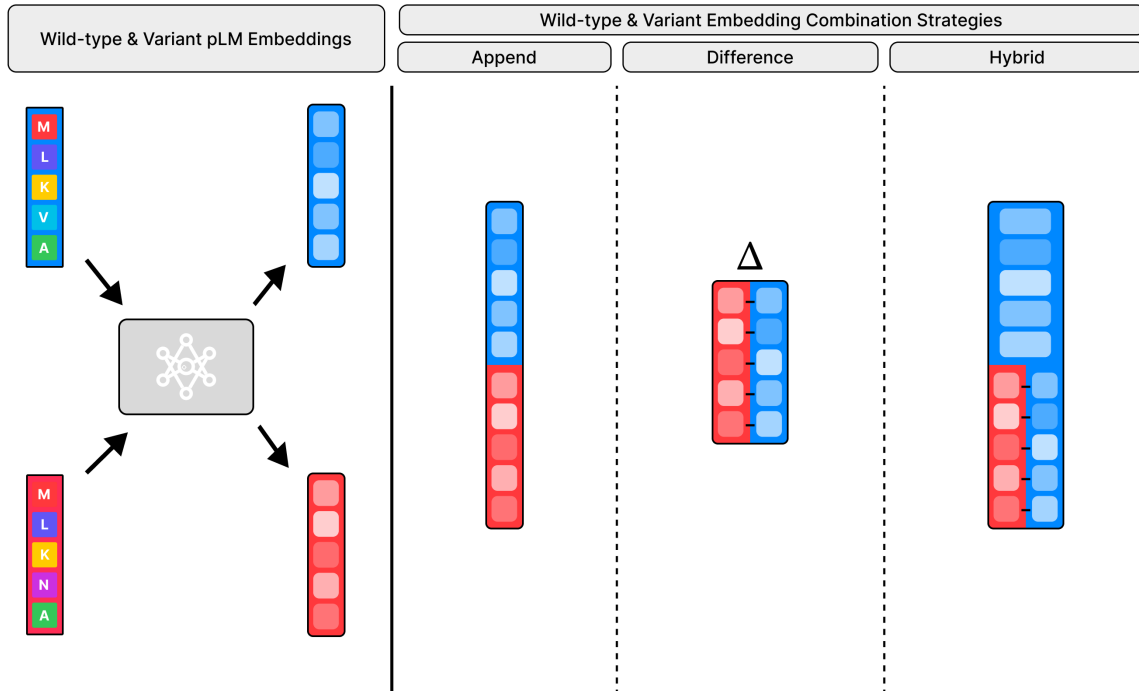


Figure 2.7: Visualization of the three wild-type & variant embedding combination strategies.

This is why in this experiment, three combination strategies for the wild-type embeddings (representing the original sequence) and the variant embeddings (representing the mutated sequence) are evaluated and compared against each other: the *append*, *difference* and *hybrid* strategies. Figure 2.7 shows a diagram visualizing the three combination strategies.

- ***Append***

In the append combination strategy, as the name suggests, we simply concatenate the embeddings of the variant after the embeddings of the wild-type. This strategy is the most conservative of the three, since it technically contains enough information to capture all relevant patterns, as both embeddings are present. One potential downside of this method is that it effectively doubles the input space dimension, which directly translates to a more complex learning problem [5].

- ***Difference***

On the other hand, the difference strategy only keeps the mathematical difference between the two embedding vectors. This approach removes any gene-level features from the input, which removes the contextual information surrounding a mutation entirely, only leaving information about the mutation itself. Compared to the *append* method, the input space is not doubled, making the learning process less complex. On top of that, removing the gene-level context information might force the learners to learn the patterns within mutations instead of learning which gene is more associated with cancer development, which is ultimately what is intended in this downstream task. However, information is still lost in this method, which may negatively affect learning performance.

- ***Hybrid***

The third approach, the hybrid combination strategy, is a combination of both: the difference of the embedding vectors is concatenated to the embeddings of the original (wild-type) sequence. This method keeps the context information and does not lose any information, while still explicitly containing the mutation information. However, similarly to the *append* strategy, the input space is still doubled, which leads to the same downsides.

To determine which combination strategy is optimal with respect to accuracy and F1-score for our next experiments, several single-pLM RF (or TL) models will be trained and evaluated three times; one for each wild-type & variant embedding combination strategy. It is worth mentioning that a single-pLM RF model and a single-pLM TL model are exactly equivalent, since the average of a single output is equal to that output. During training and testing, the errors (binary cross-entropy loss), true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN) will be measured, such that any performance metric can be computed later on.

2.6.2 Experiment 2: Individual pLM Performance

The objective of this experiment is to determine which pLMs out of the twelve create the best latent representations of proteins from their amino acid sequence, and which pLMs are not fit for the cancer driver mutation classification task.

To assess the pLMs' performance, we will be training and benchmarking the two single-model ensemble architectures (RF and TL) with one pLM for each of the twelve pLMs, as well as all pair-wise combinations of the twelve pLMs and some custom combinations. After obtaining the results, two analyses will be performed: the raw results of the single-pLM models, and the averaging of the results of all combinations in which a pLM contributed (per pLM). This enables the evaluation of a pLM's performance when evaluated individually and when evaluated in combination with others, since embedding spaces can complement, overlap or overshadow each

other. How those latent spaces interact with each other will be experimented and discussed in Experiment 4. The reason why only the single-model ensemble architectures are considered for this experiment is because the way the stacked models work may confound the analysis due to various reasons, such as the order of pLMs in the sequential model having the ability to affect the performance of the overall model.

As a baseline for this experiment, we have decided to also include a simple model that takes the raw tokenized sequences instead of a pLM latent vector as input, and have named it the “NoPlm” model. This baseline provides us with an objective performance threshold, allowing the pLM models to be systematically categorized according to whether it performed superior or inferior to a model with no protein knowledge. Consequently, pLMs that underperform a model that totally lacks protein knowledge will not be considered in the next experiments, which will ease the interpretation of the next experiments’ results.

2.6.3 Experiment 3: Ensemble Architecture Performance

In this experiment, all four ensemble architectures will be evaluated by training and benchmarking them with many pLM combinations of the best performing pLMs identified in the previous experiment. Additionally, to show that these pLMs actually output a latent representation of proteins, a model with an MLP that directly receives the encoded wild-type and variant sequences will also be trained and benchmarked. After that, we will compare the accuracies and F1-scores of each of the models, and also analyse which latent features contribute the most to the output through feature attribution analysis using the **Captum**; a model interpretability framework in PyTorch used to compute feature attribution weights. That way we can see whether the ensemble architectures combine the pLMs well or not, which will help determine which ensemble model will be used for the next and last experiment.

2.6.4 Experiment 4: Feature-Level pLM Combination Analysis

For this experiment, we used the Captum¹⁰ library to perform integrated gradients feature attribution in order to analyse the internal behaviour of the pLM combinations. We followed Captum’s official documentation¹¹ as well as PyTorch’s tutorial¹², using 50 steps and a batch size of 64 on the ALL test dataset. By default, Captum’s integrated gradients method returns a tensor containing an attribution score for each feature for every sample in the test set. Since these scores can be both positive and negative, to simplify the analysis, we define a simpler metric to quantify the overall importance of each feature:

$$\text{global_importance}(\text{attributions}) = \frac{\sum_{i=1}^D |\text{attributions}_i|}{D} \quad (2.7)$$

where D denotes the total dimensionality of the ensemble model, and attributions_i represents the attribution scores of latent feature i across all test samples. This global importance metric removes the need to distinguish between positive and negative contributions to the output, allowing us to focus on the overall influence of each feature.

This experiment focuses exclusively on the representation fusion (RF) and task-level with meta-learner (ML) ensemble architectures. The regular task-level (TL) architecture is excluded because

¹⁰<https://captum.ai/>

¹¹https://captum.ai/docs/extension/integrated_gradients

¹²<https://docs.pytorch.org/tutorials/beginner/introyt/captumyt.html>

its task learners operate in a manner equivalent to those in the single-pLM RF setup, as well as those in the ML architecture in the first phase. Furthermore, feature attribution is only meaningful for the RF task learner and the ML meta-learner, since in the TL architecture the final aggregation is performed via a non-trainable averaging operation.

The pLM combinations analysed in this experiment are selected based on the results of the previous experiments and include the best-performing pair, the worst-performing pair, and an additional representative combination. As mentioned in Section 2.5, only the persisted models from the first fold of the cross-validated pipeline are used for this analysis.

Chapter 3

Results

In this section, the results of each of the four experiments are presented and discussed. Note that generative AI tools were used to assist in the creation of some of the plots presented throughout this chapter.

3.1 Experiment 1: Wild-Type & Variant Combination Strategies

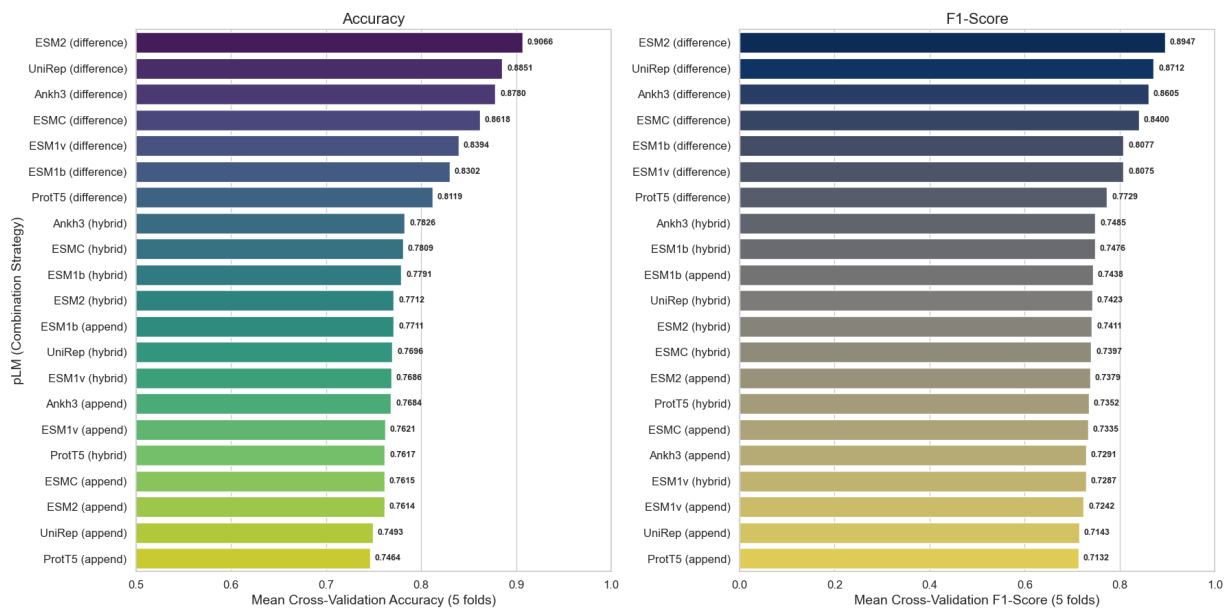


Figure 3.1: Ranked pLM with Different Wild-type & Variant Combination Strategies.

According to the performance rankings on Figure 3.1, the *difference* combination strategy consistently outperforms the alternative methods, with the *hybrid* strategy in second place, and the *append* strategy as a close third. This clear victory can also be seen on Figure 3.2 and Figure 3.3,

with the performance gap becoming larger as the accuracy/F1-score of the pLM in question increases.

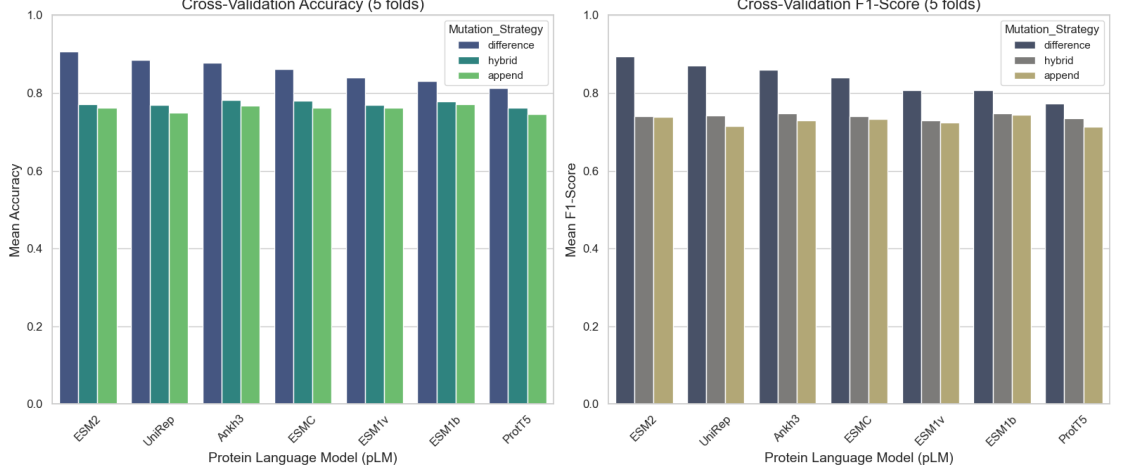


Figure 3.2: Impact of Wild-type & Variant Combination Strategies Across pLMs.

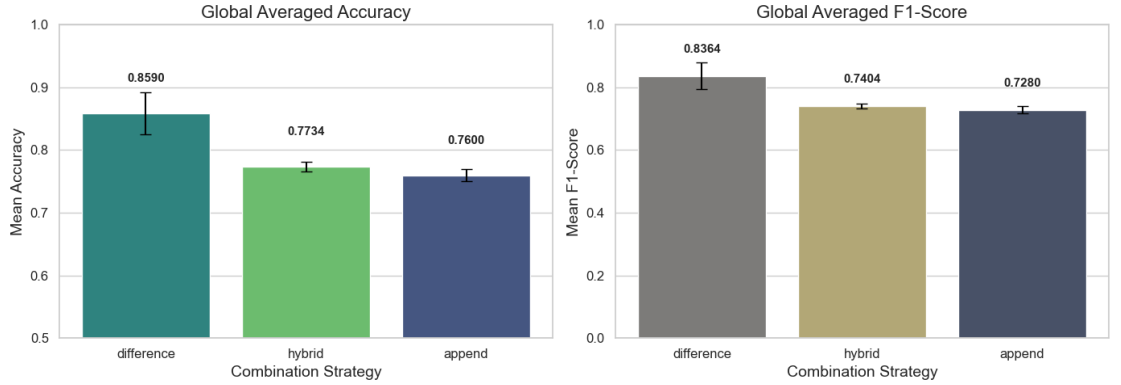


Figure 3.3: Average Impact of Wild-type & Variant Combination Strategies Across pLMs.

An explanation as to why the *difference* method performed well can be given based on its advantages, which were discussed in Chapter 2: having the model exclusively rely on the mutation information without the context data (i.e., the sequence) forces the model to learn patterns about the mutation, and not about the proteins themselves [22, 1]. Consequently, this also explains why the *hybrid* method performed (slightly) better than the *append* method while having the same input dimensionality, since it explicitly contains data about the mutation itself (rather than containing context information twice).

The superior performance of the *difference* combination method is also due to it not doubling the dimensionality of the input space, which also helps with the learning process [5].

3.2 Experiment 2: Individual pLM Performance

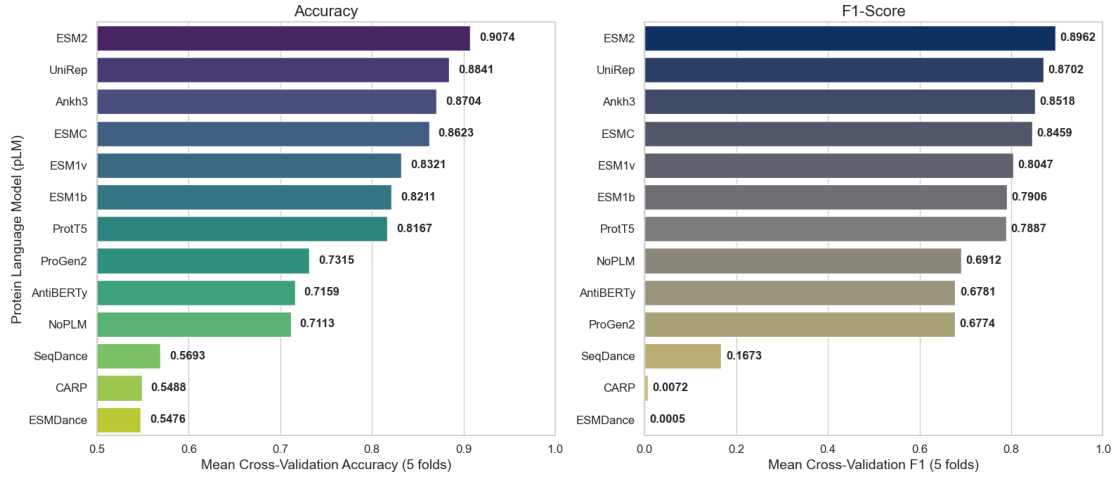


Figure 3.4: Standalone pLM Performance.

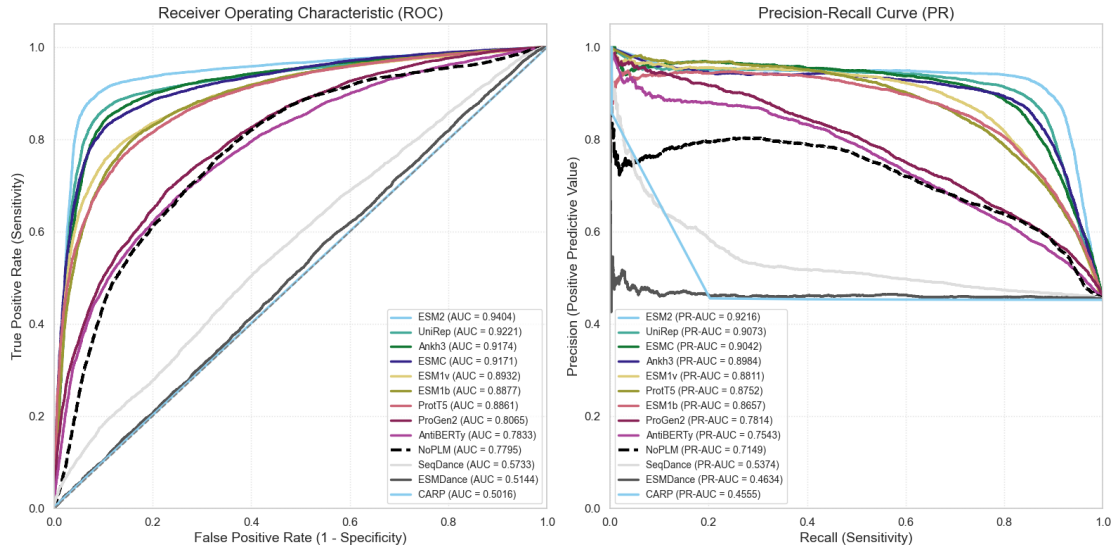


Figure 3.5: Standalone pLM Area under the Curve (AUC).

The data of the standalone pLM performance on Figure 3.4 show that the twelve pLMs can be separated into two groups: the pLMs that generate embeddings with which the MLPs almost consistently fail to predict the target, and the pLMs that learn at least meaningful latent features for the proteins. The former group includes the protein dynamics-centred SeqDance and ESM-Dance models and the CNN-based CARP model. ESM-Dance and CARP resulted in an accuracy of around 55%, which approximates the balance of the dataset, meaning the models (almost) exclusively predicted “True” for every input sample. This phenomenon can be further evidenced

by the F1-scores being very close to zero. SeqDance, however, did demonstrate marginal predictive capacity, proving that it did learn some latent space of proteins, but not an optimal one.

The pLMs that did effectively learn meaningful features can be subdivided into two groups based on predictive performance of the attached MLP: ProGen2 and AntiBERTy, which performed sub-optimally compared to the other pLMs, and the remaining others. AntiBERTy is a rather small model of 26M parameters [33] compared to the similarly performing ProGen2 model, which has 6.4B parameters [36]. The unexpected underwhelming performance of the rather large ProGen2 model, however, can be explained by its architecture. Since it is an autoregressive model, in contrast to the masked pLMs that trained on randomly masked sequences, ProGen2 has only been trained to predict the next residue given a sequence. ProtT5, a well-known state-of-the-art model, unexpectedly returned disappointing results and got outperformed by the much smaller ESM-1b model (3B parameters versus 650M; [7, 8]). A plausible explanation for this is that ProtT5 was trained for very different tasks, or that the DRGN dataset is simply an outlier for it. The remaining pLMs, with ESM-2, UniRep and Ankh3 being the top three, performed rather well on their own in the cancer driver mutation prediction task, having accuracies ranging from 70% to 90%

We now discuss the results from the perspective of pLM categories. The general-purpose pLMs include the high-performing ESM-2, UniRep, Ankh3, ESM-C, ESM-1b, and ProtT5, as well as the unexpectedly underperforming ProGen2 and CARP. Overall, the general-purpose models outperformed all subdomain-specific pLMs (AntiBERTy, SeqDance, and ESM Dance), with the notable exception of ESM-1v, a variant-focused model derived from ESM-1b through additional fine-tuning on protein variants.

In contrast, AntiBERTy was trained specifically on antibody sequences, while ESM Dance and SeqDance were developed for protein dynamics tasks. This specialization likely contributed to their comparatively poor performance on the cancer driver mutation classification task, as the knowledge learned during pre-training is less aligned with the requirements of the downstream task. This means that the different latent spaces of those models do not capture useful information for this particular task.

What was unexpected was for ESM Dance to underperform SeqDance, since the main difference between the two pLMs are the fact that ESM Dance started with initial weights from the pre-trained ESM-2.35M model, while SeqDance was randomly initialized. A possible explanation for this is that the initial ESM-2 weights might have been too far from the optimal weights due to ESM-2 not being trained on protein dynamics data.

If we consider the macro contribution of each pLM in different pairs as illustrated in Figure 3.6, a similar trend can be seen, as well as a new interesting trend: all models of the ESM-family are at the very top of the scores. This would mean that they complement the input spaces of the pLM they’re paired with no matter which pLM they’re working with, which implies that the ESM family of models learned a very diverse and wide range of latent features. This is essentially the best quality a general-purpose model can have, and the recipe to high performance on downstream tasks.

Another new trend that can be seen on Figure 3.6, is that the best performing models performed almost identically on both the RF and the TL ensemble architecture, but the worst performing pLMs performed better on the TL architecture compared to the RF ensemble on average. That is a direct advantage of the decision-level ensemble models: instead of adding extra input dimen-

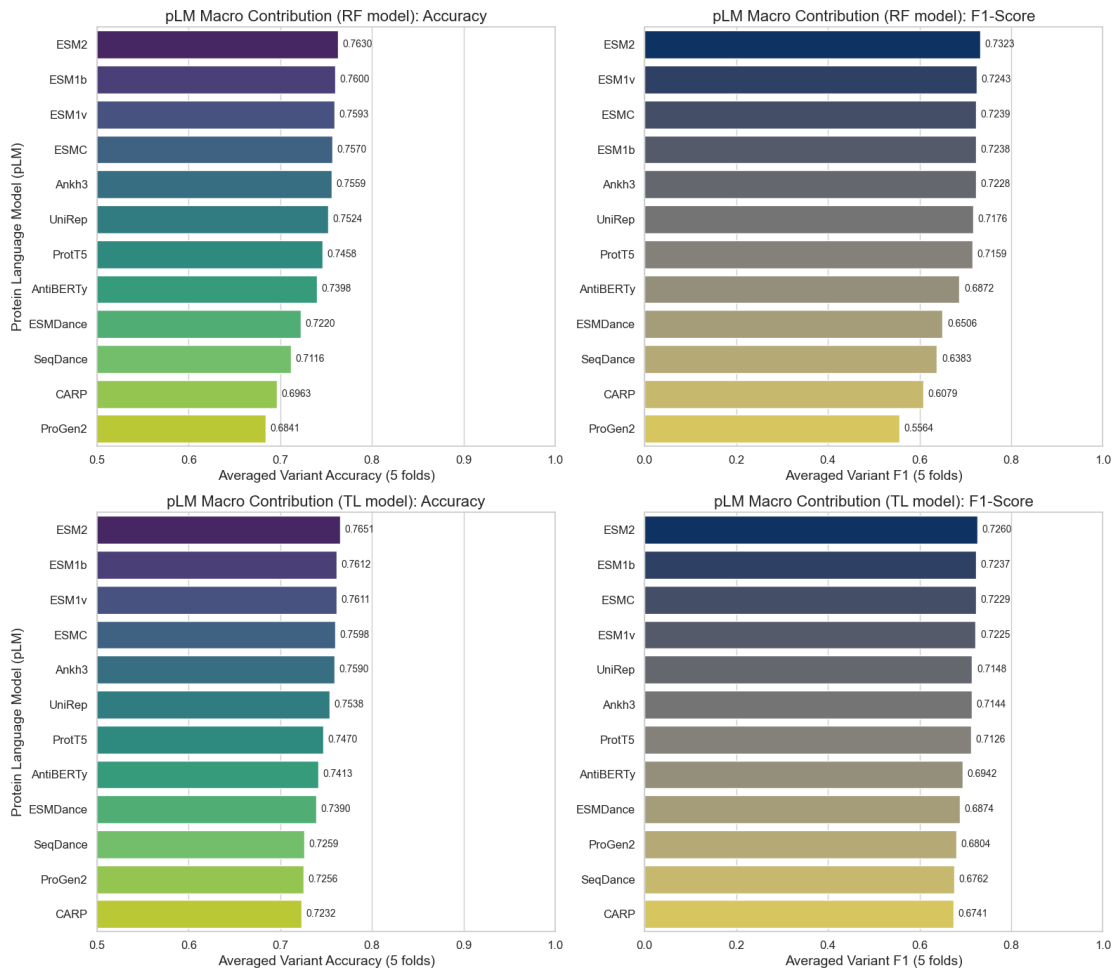


Figure 3.6: pLM Macro Contribution Across All Ensemble Pairings, for both the RF and TL architectures.

sions that might distract or mislead the MLP, the TL architecture only has to average the output of the independently-trained inner task learners, circumventing the dimensionality problem.

When considering the ROC-AUC plot in Figure 3.5, the performance ranking between the pLMs largely stays unchanged, with the underperforming group that was discussed earlier showing a curve closely following the diagonal reference line and having AUC scores close to 0.50. This indicates that these models are nearly as likely to predict a higher score for a negative sample as to a positive one, meaning that their ability to distinguish between the two classes is close to random. The group of optimally performing pLMs show curves closer to the top-left of the ROC-AUC plot (and the top-right of the PR-AUC plot), while having scores roughly between 0.80 and 0.95. This confirms that those models effectively generate embeddings with latent features that represent additional protein-specific properties that can not be derived from the flat sequences alone.

As mentioned in Chapter 2, the pLMs that did not perform at least as well as the “NoPLM”

model will be considered not fit to generate correct features for this cancer driver mutation prediction task. These include ESM2Dance, CARP, SeqDance, ProGen2 and AntiBERTy.

3.3 Experiment 3: Ensemble Architecture Performance

According to the results, which can be found on Figures 3.7 and 3.8 for the representation fusion model (RF), Figures 3.9 and 3.10 for the task-level model (TL) as well as Figures 3.11 and 3.12 for the task-level with meta-learner model (ML), the RF model seems to have the best ROC-AUC and PR-AUC values, and the best lower-bound ROC-AUC and PR-AUC values. This suggests that the RF model can better distinguish positive samples from negative samples compared to the TL and ML models. From these results, it appears that the RF model extracts the strongest signal from the combined embedding space. A plausible hypothesis that may explain this phenomenon is the fact that, as the RF task learner MLP can access all embeddings at once, allowing it to leverage the information contained within the combined embedding space, while the task learners in the TL and ML architectures (excluding ML’s meta-learner) can only work with the embedding space of their respective pLM.

However, processing all the protein data at once also has its downsides (i.e., the curse of dimensionality [5], as mentioned in Chapter 2), which may be reflected by the better accuracies and F1-scores of the ML ensemble model, with the TL model being a close second. Additionally, the TL model had the highest lower-bound accuracy and F1-scores of the three models, which was surprising. We originally thought that, since it lacked the ability to give lower weights to less important pLM latent features, it would be negatively impacted in performance e.g., in case a pLM with exceptional individual performance was combined with a pLM with suboptimal individual performance. Instead, despite that fact, the results show that imbalanced (with respect to individual performance) pairs such as CARP+ESM2 and Ankh3+SeqDance are listed among the top performers in the accuracy and F1-scores leaderboards. A similar trend can be found in the results of the ML models (see Figure 3.11), in which for instance ESM2 paired with SeqDance is among the best combinations, even though SeqDance was one of the worst-performing models in the individual pLM analysis in the previous experiment. This can be explained by the fact that SeqDance’s latent space exclusively captures protein dynamics information, which complemented ESM-2’s general-purpose nature well. One possible hypothesis is that combining the task outputs of many independent MLPs with their respective pLM’s latent space results in a reduction in prediction variance due to compensating errors of every independent models, making them cancel each other out.

In this experiment, we have found that the performance of the ensembles consistently exhibited underwhelming performance compared to the single-pLM counterparts, meaning that our objective of outperforming single-pLM models was not achieved. Potential threats to validity include model overfitting, noise accumulation during combination of results and embeddings, increased model and optimization complexity, use of the same hyperparameters for all model configurations or even slight implementation errors on our part. These possible causes might also explain the small differences in performance across ensemble architectures. On the other hand, it may be possible that the added model and optimization complexity outweighs the performance gain from the additional protein information.

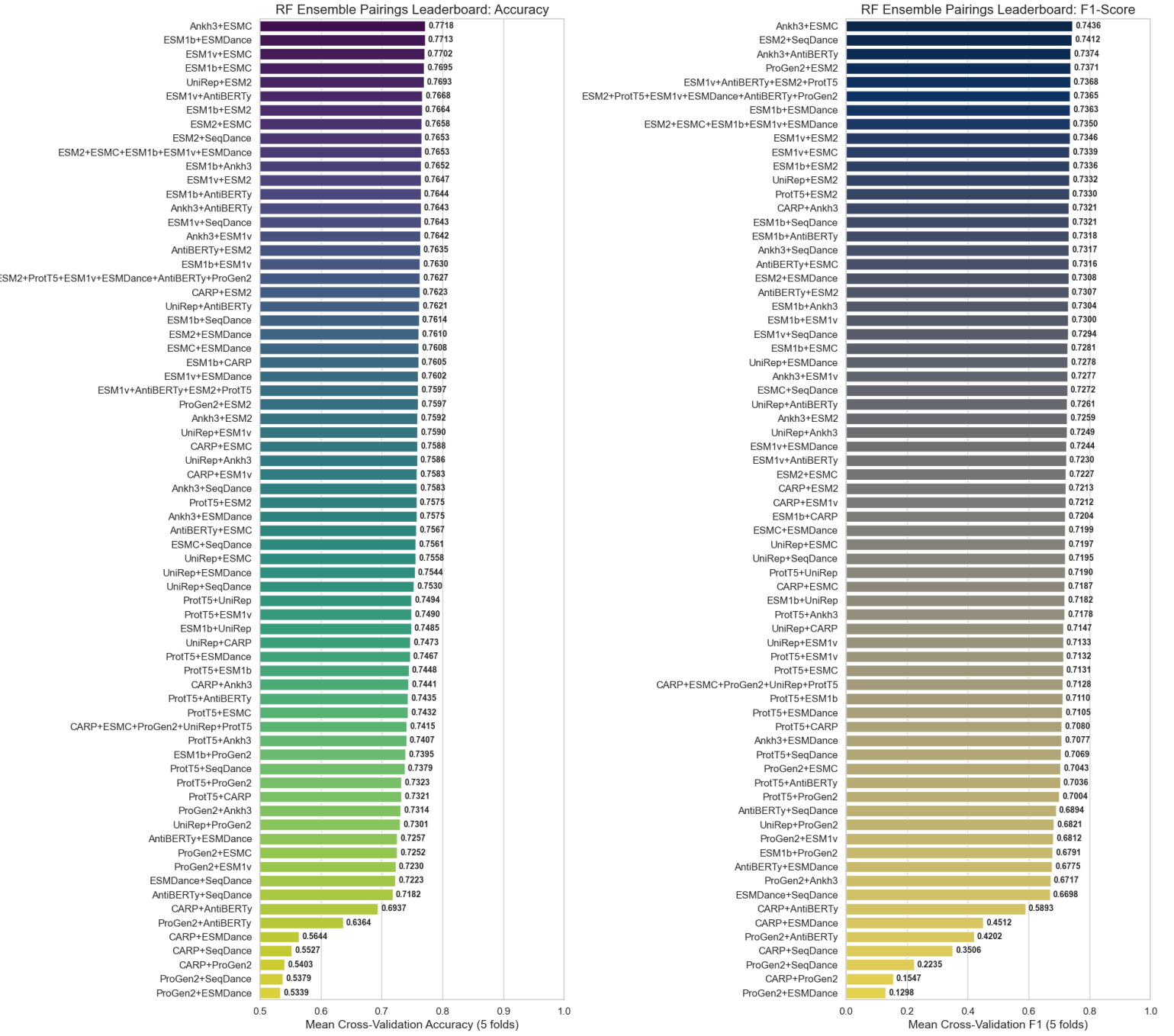


Figure 3.7: Performance per pLM Combination (RF model).

3.4 Experiment 4: Feature-Level pLM Combination Analysis

The results of this Experiment will be discussed as follows: for each of the three pLM combinations, the global feature attribution results of the representation fusion (RF) model will first be

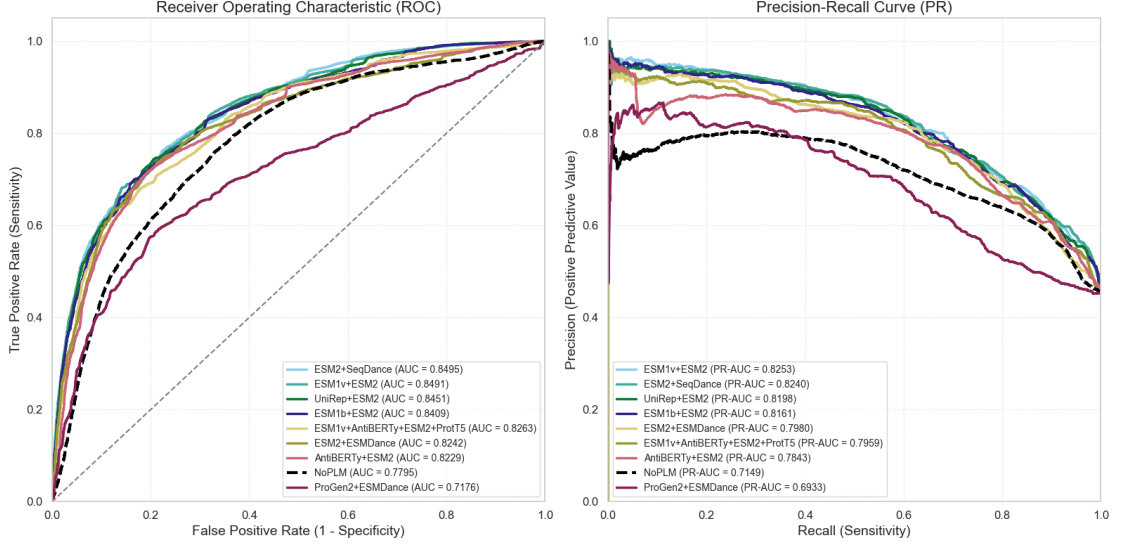


Figure 3.8: Area under the Curve (AUC) per pLM Combination (RF model).

analysed, followed by those of the task-level ensemble with meta-learner (ML) model. It is worth noting that, in the figures corresponding to the RF model, the order of the pLM pairs reflects the order in which the embedding vectors were concatenated.

The first pair of pLMs that will be discussed is ESM-2 and ESM-1v, which is one of the best-performing pLM combinations identified in Experiment 3. The global feature attribution scores can be found in Figure 3.13 for the representation fusion (RF) model and in Figure 3.14 for the meta-learner (ML) model.

As can be seen in Figure 3.13, the ESM-1v latent features exhibit lower global importance scores when combined with ESM-2 than in the corresponding single-pLM model, and even more so when compared to the ESM-2 latent features. Additionally, the figure shows that fewer ESM-2 features stand out as substantially more important than the others compared to the ESM-2-only model. These observations may be explained by ESM-1v’s features taking over part of the predictive contribution of ESM-2 features due to overlap in both latent spaces. This hypothesis is plausible given that ESM-2 is essentially a scaled-up successor of ESM-1b, which serves as the base model for the fine-tuned ESM-1v model [30].

Figure 3.13c, which shows the per-feature global importance scores of the ESM-1v-only model, also exhibits substantially higher attribution scores than the ESM-2-only model shown in Figure 3.13b. This difference is likely caused by the same number computation steps during the feature attribution for both single-pLM models despite their substantially different embedding dimensions (5120 and 1280 for ESM-2 and ESM-1v, respectively). However, this does not affect the validity of our analysis, since we focus on relative changes in feature importance rather than absolute attribution scores.

For the task-level with meta-learner architecture, Figure 3.14 shows that the output of the MLP trained on ESM-2 embeddings is assigned greater importance by the meta-learner than the out-

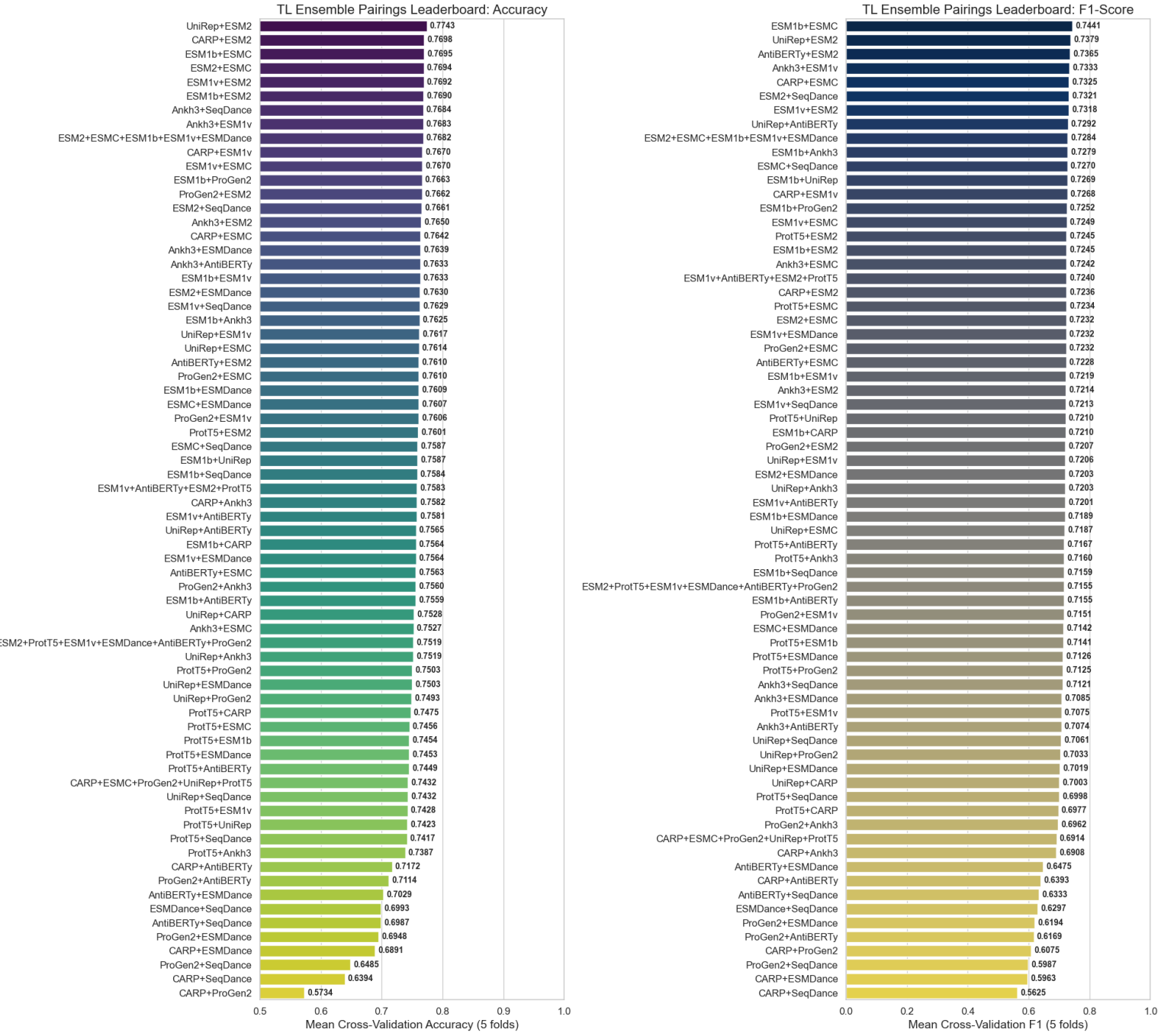


Figure 3.9: Performance per pLM Combination (TL model).

put of the MLP trained on ESM-1v embeddings. This suggests that, for our downstream task, ESM-2 provides more informative latent representations than ESM-1v. This observation is not surprising given the substantial differences in parameter count and training data between the

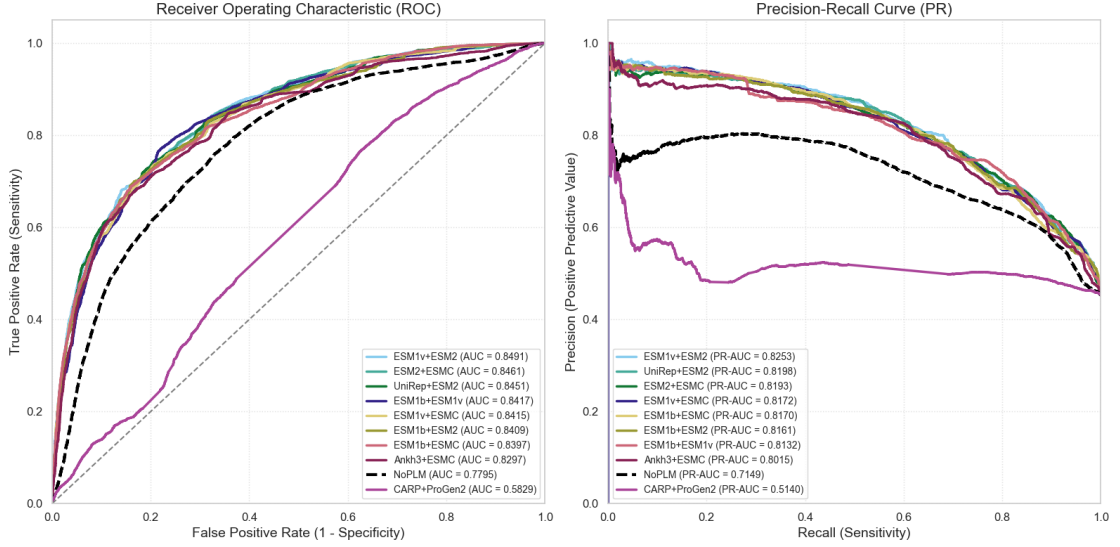


Figure 3.10: Area under the Curve (AUC) per pLM Combination (TL model).

two models.

We will now discuss the feature attribution results concerning one of the worst-performing pLM pairs, namely CARP and ProGen2. As can be seen in Figure 3.15a, the contribution of the CARP embeddings is practically nonexistent, while the ProGen2 portion of the fused feature space is used much more extensively. In other words, the RF task learner appears to rely almost exclusively on ProGen2’s latent features, assigning very little importance to the latent features generated by CARP. Additionally, the distribution of per-feature global importance scores for ProGen2 in the combined model is nearly identical to that of its single-pLM counterpart. These findings suggest that CARP captures little useful information to the cancer driver mutation prediction task within this particular combination. As a result, CARP may increase the dimensionality of the input space without providing any performance benefit, which could partially explain why this pair is among the worst-performing combinations in our experiments.

It is worth mentioning that CARP exhibits the sparsest per-feature global importance distribution of all analysed pLMs, with almost all features having negligible importance scores except for a few features (see Figure 3.15b). A potential explanation is that the protein properties captured by CARP’s underlying CNN architecture are highly specialized and therefore less relevant to the cancer driver mutation prediction task. Note that, because the importance scores are substantially lower than those of the other pLMs, the scale of the y-axis was adjusted to make these differences more visible.

The results of the meta-learner architecture in Figure 3.16 reveal an interesting observation: the meta-learner assigns greater importance to CARP’s task learner in the pLM-pair model than in the single-pLM CARP model. This may suggest that the information captured by CARP becomes more useful when combined with the information captured by ProGen2. However, since this is a decision-level ensemble architecture, these results can not directly prove the interactions between the underlying embedding spaces. Therefore, it cannot be concluded from these

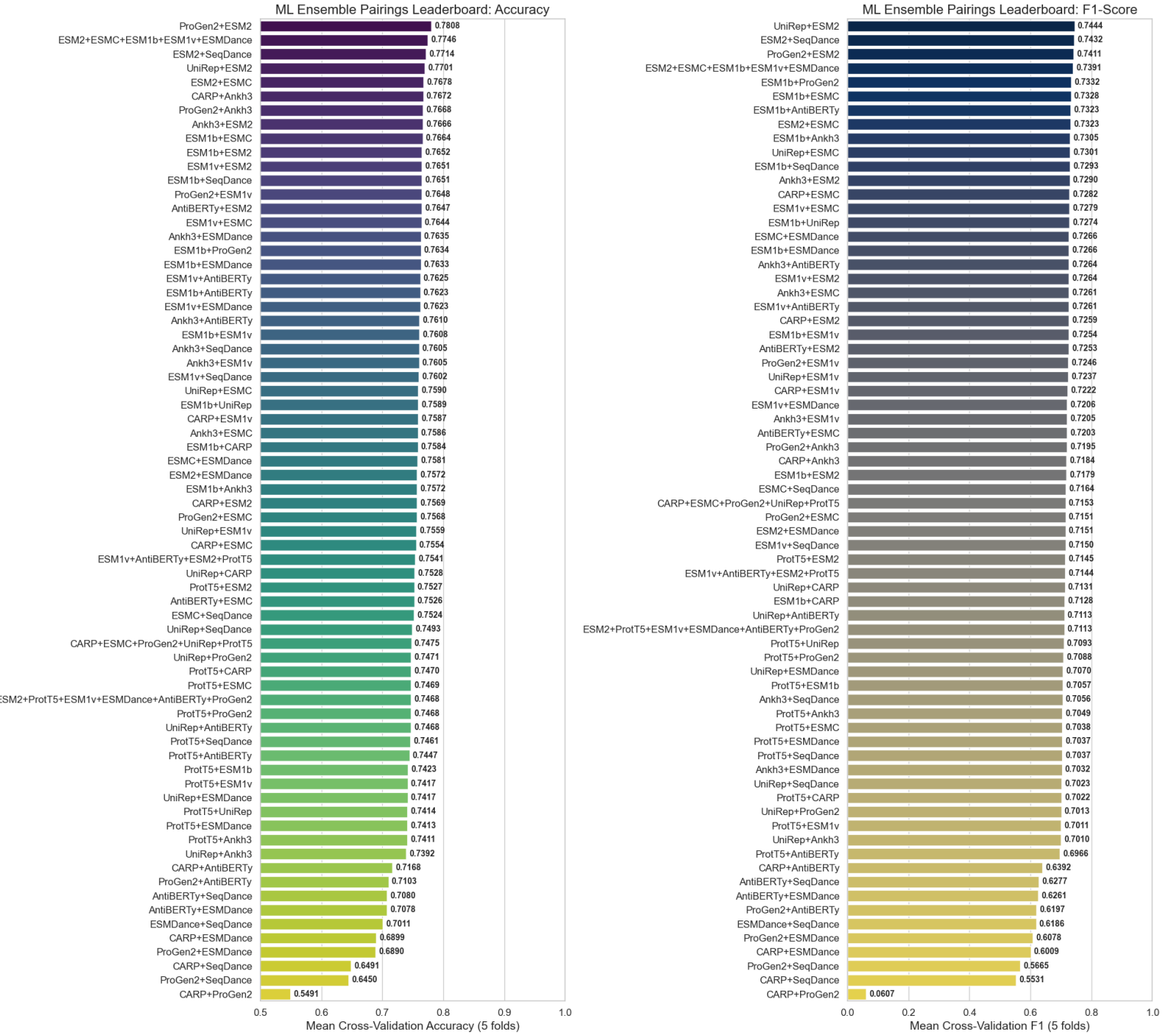


Figure 3.11: Performance per pLM Combination (ML model).

results alone whether the latent representations of CARP and ProGen2 are truly complementary.

Finally, we will discuss a well-performing and interesting pair from Experiment 3: ESM-2, a

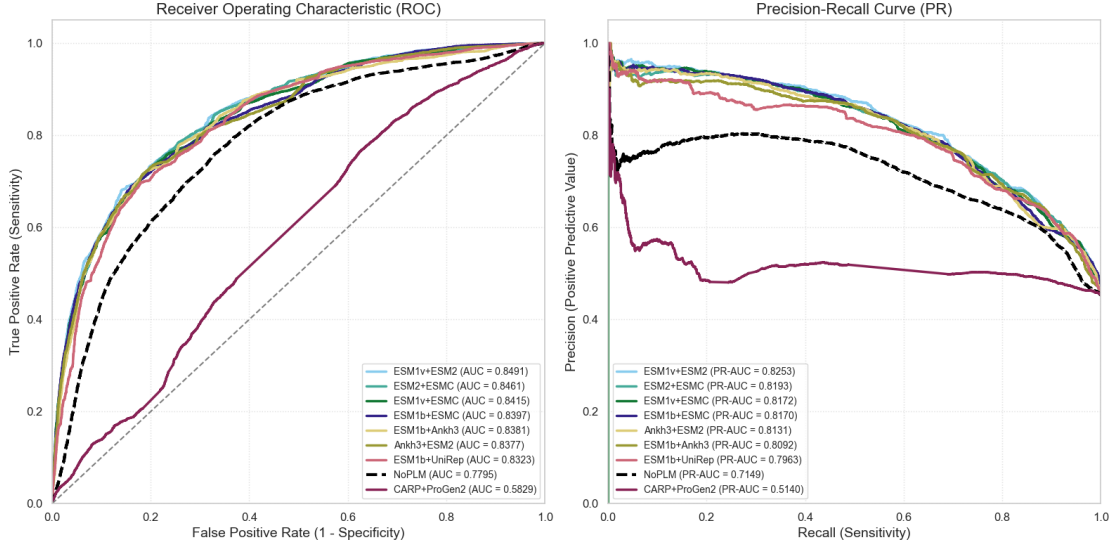
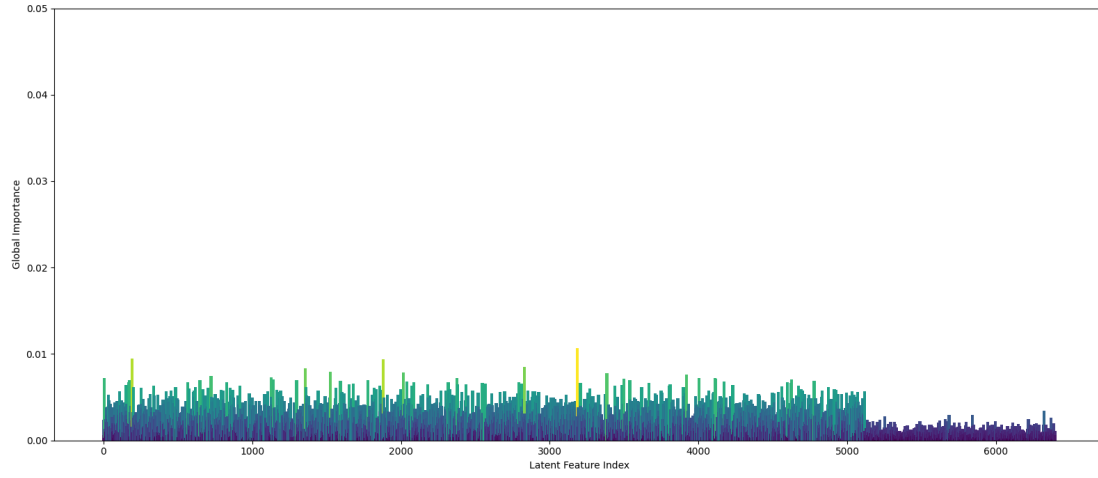


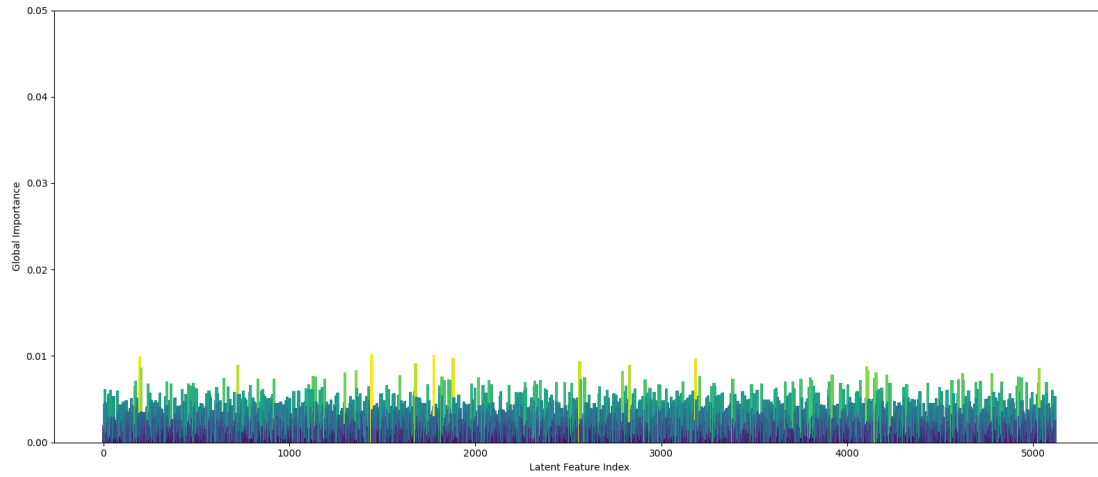
Figure 3.12: Area under the Curve (AUC) per pLM Combination (ML model).

large general-purpose pLM, and SeqDance, a smaller protein dynamics-focused pLM. This combination is particularly interesting because the ESM-2+SeqDance pair displays a similar pattern to the ESM-2+ESM-1v pair, where using the second model changes per-feature importance distribution of ESM-2’s embeddings, despite SeqDance having one of the weakest single-pLM performances (see Figure 3.17). In the case of ESM-1v, which performs relatively well on its own, its embeddings appear to overlap more strongly with those of ESM-2, whereas SeqDance appears to complement it in captured information. This suggests that the additional protein dynamics information may help the model better leverage ESM-2’s feature space, which indicates that protein dynamics-related representations could be relevant for protein point-mutation prediction tasks.

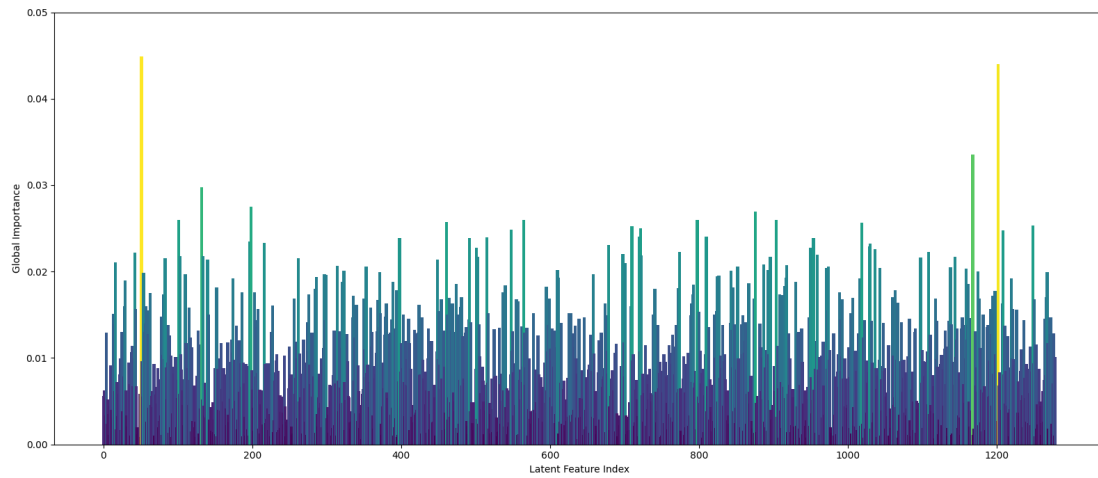
The importance scores of the ML model (Figure 3.18) are similar to those observed in the case of the ESM-2+ESM-1v pair. The only difference is that the global importance scores of the SeqDance-only model are much lower than those of ESM-2 and ESM-1v, which is consistent with the observations outlined above. Apart from that, no further conclusions can be drawn from this limited information.



(a) ESM-2 + ESM-1v



(b) ESM-2 only



(c) ESM-1v only

Figure 3.13: Integrated Gradient Feature Attribution Landscape for the Representation Fusion Architecture using ESM-2 and ESM-1v embeddings.

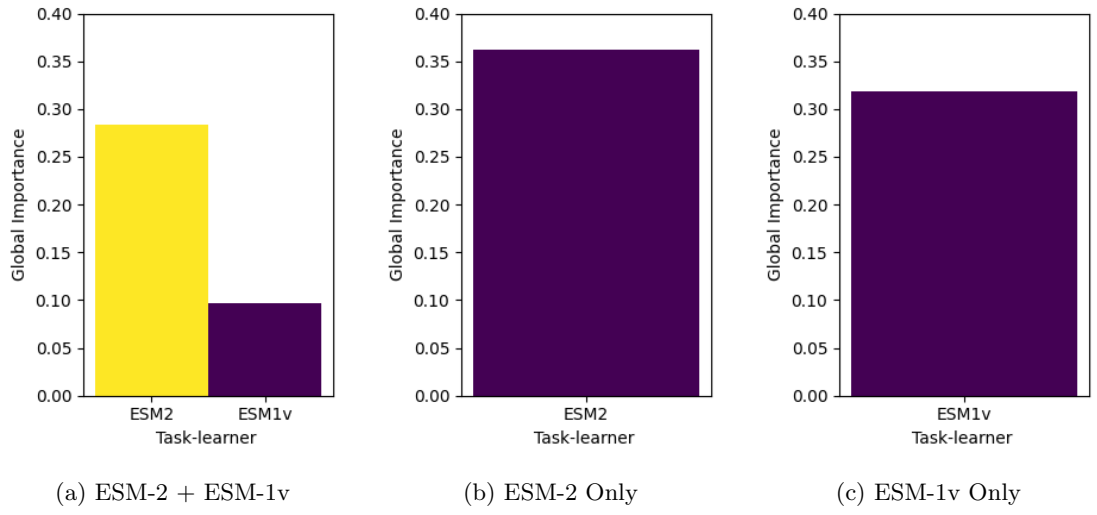
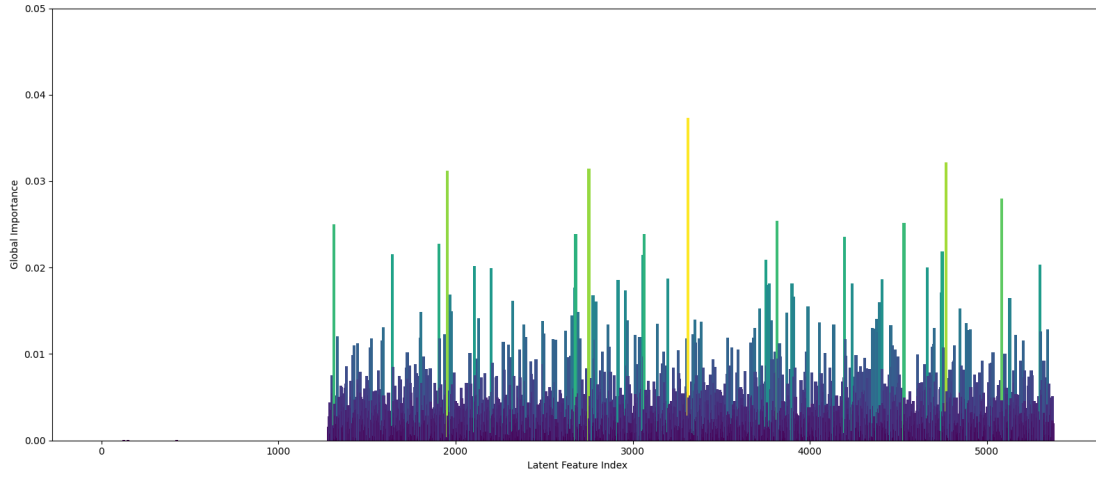
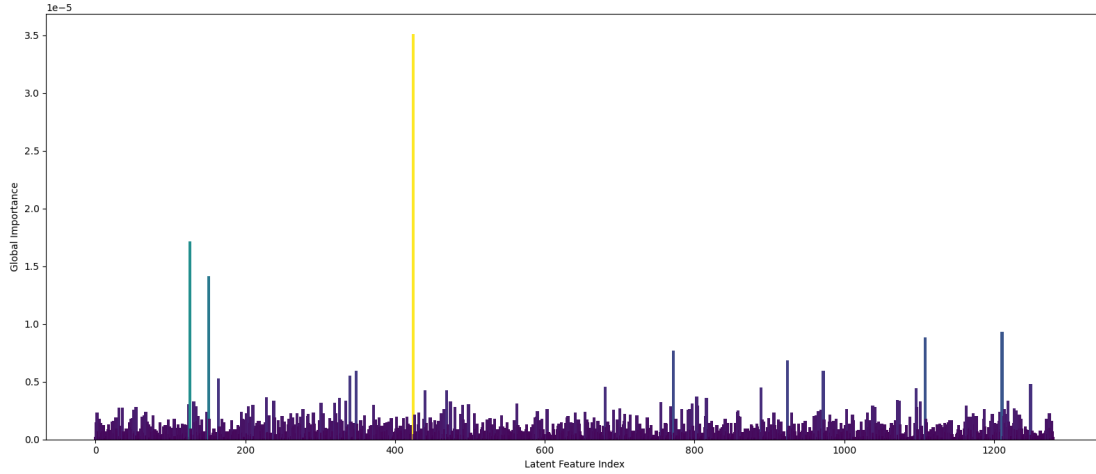


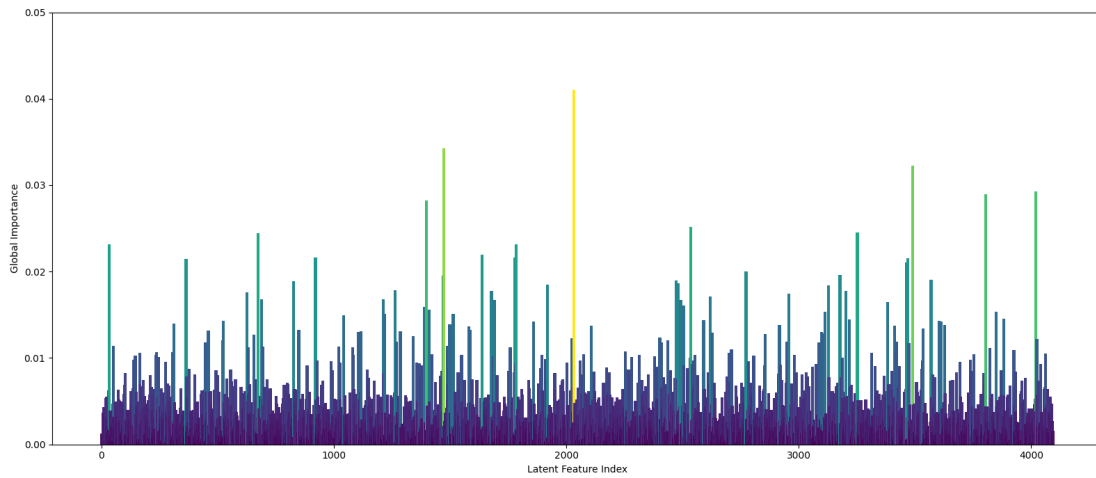
Figure 3.14: Integrated Gradient Feature Attribution of the Task-level with Meta-learner Model with pLMs ESM-2 and ESM-1v.



(a) CARP + ProGen2



(b) CARP only (scaled for visibility)



(c) ProGen2 only

Figure 3.15: Integrated Gradient Feature Attribution Landscape for the Representation Fusion Architecture using CARP and ProGen2 embeddings.

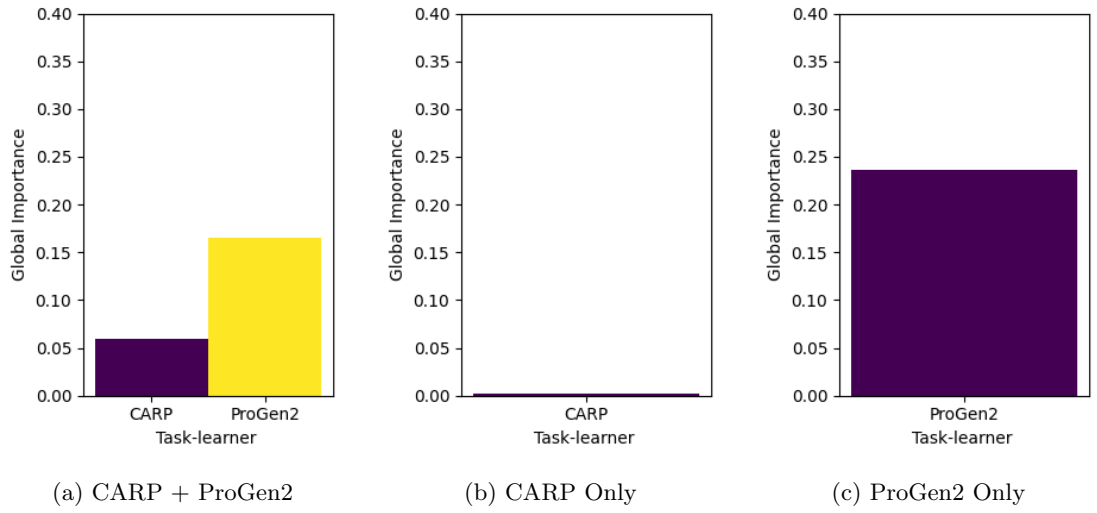
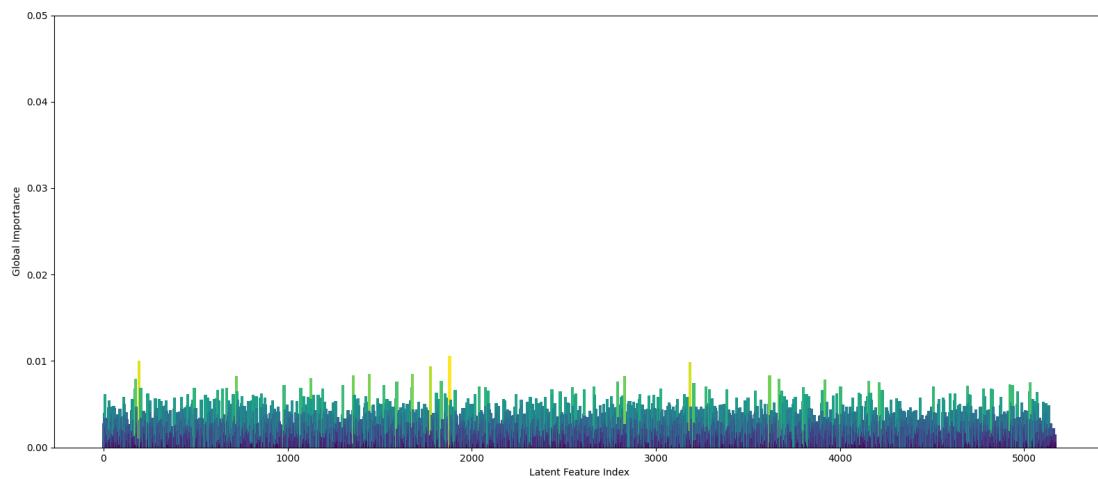
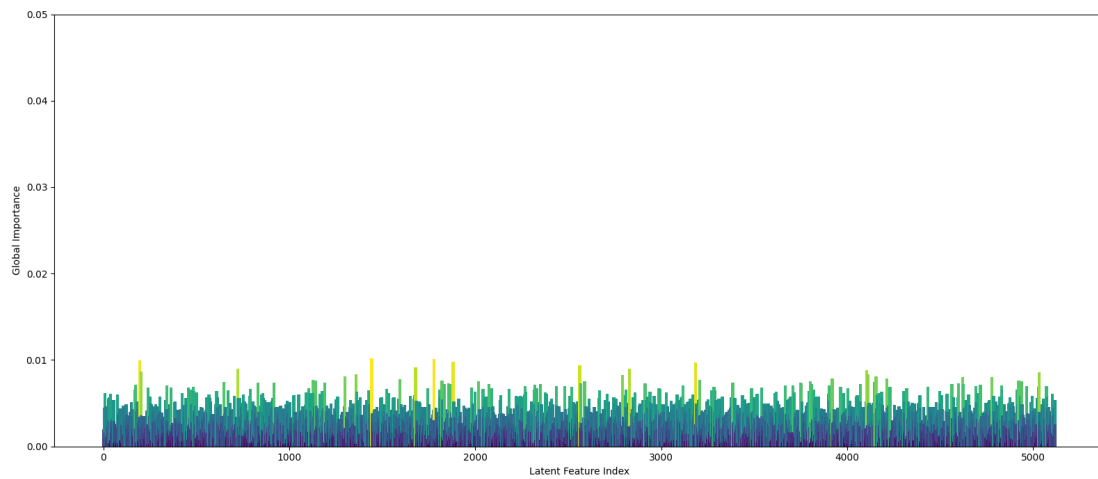


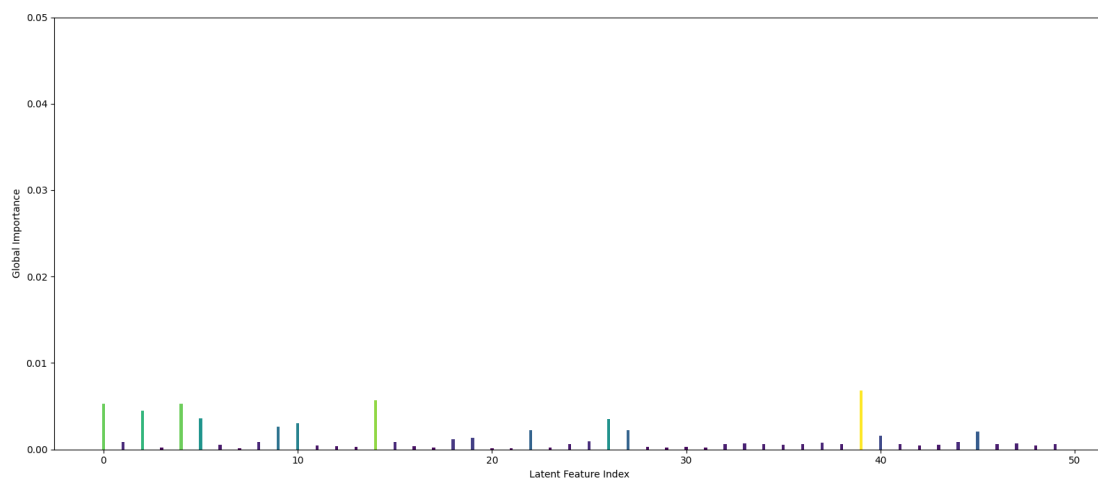
Figure 3.16: Integrated Gradient Feature Attribution of the Task-level with Meta-learner Model with pLMs CARP and ProGen2.



(a) ESM-2 + SeqDance



(b) ESM-2 only



(c) SeqDance only

Figure 3.17: Integrated Gradient Feature Attribution Landscape for the Representation Fusion Architecture using ESM-2 and SeqDance embeddings.

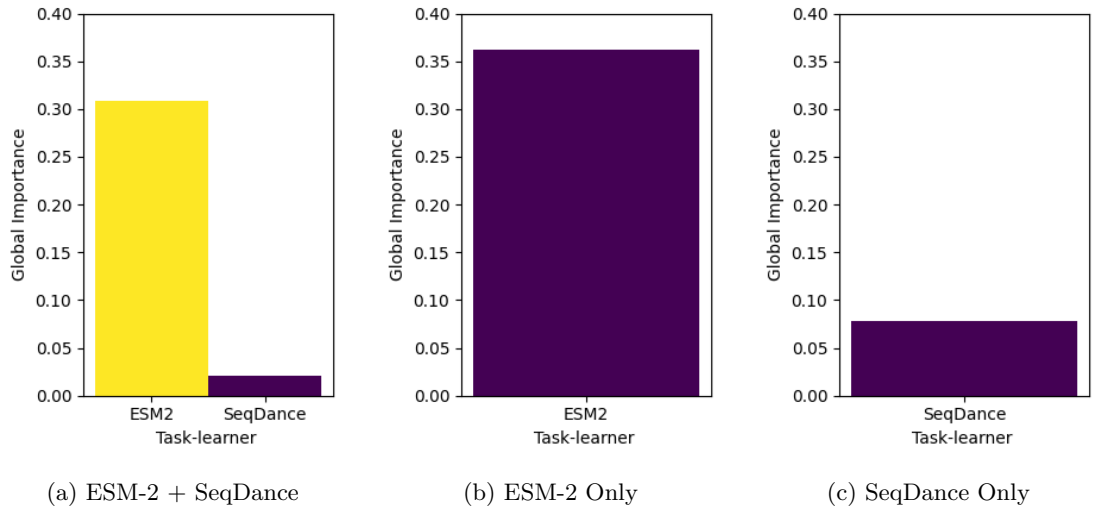


Figure 3.18: Integrated Gradient Feature Attribution of the Task-level with Meta-learner Model with pLMs ESM-2 and SeqDance.

Chapter 4

Conclusion

The results of the first experiment suggest that the “difference” wild-type & variant combination strategy is superior to the “append” and “hybrid” methods, despite lacking explicit sequence context information. This may be attributed to the fact that it only contains information about the mutation itself, thereby forcing the model to learn mutation-specific patterns rather than patterns related to a protein’s general tendency to contribute to cancer development. Additionally, unlike the other two strategies, the “difference” approach does not double the input dimensionality of the model, which may also contribute to its superior predictive performance.

Secondly, the individual pLM performance evaluation suggests that newer and larger pLMs, such as ESM-2, UniRep, Ankh3, and ESM-C, achieved the best overall performance, whereas more domain-specific models such as ESM-Dance and SeqDance, as well as general-purpose models such as ProGen2 and CARP, demonstrated substantially lower predictive performance. The latter group of pLMs was even outperformed by a pLM-less baseline model trained solely on tokenized protein sequences, without access to any additional biological knowledge encoded by protein language models.

The third experiment suggests that no single ensemble architecture consistently outperformed the others across all evaluation metrics. The representation fusion ensemble model demonstrated a stronger ability to distinguish positive samples from negative samples, suggesting that simultaneously leveraging information from multiple latent spaces may improve class separability. On the other hand, combining the predictions of multiple independent task learners, each operating on its own pLM latent space, appeared to benefit the task-level approaches. This effect was particularly noticeable for the meta-learner architecture, which achieved the highest accuracies and F1-scores. One possible explanation is that combining multiple independent predictors reduces prediction variance by allowing errors made by individual predictors to compensate for one another. The superior performance of the meta-learner compared to the standard task-level model may further be explained by its ability to learn how much weight should be assigned to each predictor, whereas the task-level model combines all predictors equally.

In the fourth and final experiment, integrated gradient feature attribution allowed us to analyse how the interactions between the latent spaces of combined pLMs varied depending on the specific pair. Some pLM combinations appear to provide complementary information in their embedding spaces, whereas other pairs may contain redundant information or may lead the task learner to largely ignore one of the pLMs.

However, none of the ensemble architectures or pLM combinations managed to outperform the best single-pLM models. The most probable hypotheses for this phenomenon is that the drastic increase in latent space dimensionality, architectural and optimization complexity severely outweighs the potential performance gain it could deliver. Several other factors may have contributed to this outcome include the accumulation of noise originating from the individual pLM outputs, the use of identical hyperparameter settings across all experiments, possible model overfitting and minor implementation errors, although no evidence was found to conclusively support either hypothesis.

Since the goal of this thesis was to analyse the ensemble architectures on the DRGN dataset specifically, these observations may not be representative of other datasets and should therefore be interpreted with caution. To better assess the generalizability of the proposed ensemble architectures, they should be evaluated on a wider range of downstream tasks and/or standardized benchmarking frameworks such as TAPE [37] and FLIP [38].

Chapter 5

Future Work

Aside from the gradient boosting ensemble architecture that we were unable to fully implement within the available time, there are several additional experiments that would be interesting to explore in future work:

- **Extending the ensemble models to support per-residue embeddings:** Our models currently only support per-sequence embeddings, whereas most pLMs output per-residue embeddings. It would be interesting to investigate how incorporating this additional dimension of protein information affects the downstream classification task. However, this extension may require more specialized architectures and evaluation strategies given the increased complexity of the resulting representations.
- **Cherry-picking important pLM latent features:** Throughout our experiments, we observed that larger latent dimensionalities tend to increase the complexity of the learning problem, negatively affecting predictive performance. Since the results of the fourth experiment suggest that only a subset of latent features contributes substantially to the model’s predictions, it would be interesting to investigate whether selecting the top- K most important latent features could reduce dimensionality while preserving most of the relevant information.
- **Using pLM benchmarking frameworks for inter-subdomain performance assessment:** Although this work primarily focuses on predictive performance for the cancer driver mutation classification task, benchmarking frameworks such as FLIP [38] and TAPE [37] provide standardized evaluations across multiple bioinformatics tasks and subdomains. Applying such benchmarks could help assess whether different pLMs capture complementary or task-specific biological patterns across different domains.
- **Comparing and analysing newer pLM ensemble approaches:** As mentioned in Chapter 1, several recent studies such as Venus-MAXWELL [23] and VarDCL [24] have proposed new methods for combining protein language models, as well as integrating additional sources of protein information, to improve downstream predictive performance. Applying the experiments and analyses presented in this thesis to these newer ensemble architectures could provide further insights into how pLM embeddings can be combined most effectively.

Chapter 6

References

1. Raimondi D, Passemiers A, Fariselli P, and Moreau Y. Current cancer driver variant predictors learn to recognize driver genes instead of functional variants. *BMC biology* 2021; 19:3
2. Huang B, Kong L, Wang C, Ju F, Zhang Q, Zhu J, Gong T, Zhang H, Yu C, Zheng WM, and Bu D. Protein Structure Prediction: Challenges, Advances, and the Shift of Research Paradigms. *Genomics, Proteomics & Bioinformatics* 2023 Mar; 21:913–25. DOI: 10.1016/j.gpb.2022.11.014. eprint: https://academic.oup.com/gpb/article-pdf/21/5/913/57589721/gpb_21_5_913.pdf. Available from: <https://doi.org/10.1016/j.gpb.2022.11.014>
3. Morris R, Black KA, and Stollar EJ. Uncovering protein function: from classification to complexes. *Essays in Biochemistry* 2022 Aug; 66:255–85. DOI: 10.1042/EBC20200108. eprint: <https://portlandpress.com/essaysbiochem/article-pdf/66/3/255/935846/ebc-2020-0108c.pdf>. Available from: <https://doi.org/10.1042/EBC20200108>
4. Banerjee A, Saha S, Tvedt NC, Yang LW, and Bahar I. Mutually beneficial confluence of structure-based modeling of protein dynamics and machine learning methods. *Current Opinion in Structural Biology* 2023; 78:102517. DOI: <https://doi.org/10.1016/j.sbi.2022.102517>. Available from: <https://www.sciencedirect.com/science/article/pii/S0959440X22001968>
5. Goodfellow I, Bengio Y, and Courville A. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016
6. Mitchell TM. *Machine learning*. McGraw-hill New York, 1997
7. Elnaggar A, Heinzinger M, Dallago C, Rehawi G, Wang Y, Jones L, Gibbs T, Feher T, Angerer C, Steinegger M, et al. ProtTrans: toward understanding the language of life through self-supervised learning. *IEEE transactions on pattern analysis and machine intelligence* 2021; 44:7112–27
8. Rives A, Meier J, Sercu T, Goyal S, Lin Z, Liu J, Guo D, Ott M, Zitnick CL, Ma J, et al. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the national academy of sciences* 2021; 118:e2016239118
9. Taimeskhanov M and Garreau D. Feature Attribution from First Principles. *arXiv preprint arXiv:2505.24729* 2025

10. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, and Liu PJ. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 2020; 21:1–67
11. Chen S and Guo W. Auto-Encoders in Deep Learning—A Review with New Perspectives. *Mathematics* 2023; 11. DOI: 10.3390/math11081777. Available from: <https://www.mdpi.com/2227-7390/11/8/1777>
12. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, and Polosukhin I. Attention is all you need. *Advances in neural information processing systems* 2017; 30
13. Devlin J, Chang MW, Lee K, and Toutanova K. Bert: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019 :4171–86
14. Hochreiter S and Schmidhuber J. Long short-term memory. *Neural computation* 1997; 9:1735–80
15. Krause B, Lu L, Murray I, and Renals S. Multiplicative LSTM for sequence modelling. *arXiv preprint arXiv:1609.07959* 2016
16. Alley EC, Khimulya G, Biswas S, AlQuraishi M, and Church GM. Unified rational protein engineering with sequence-based deep representation learning. *Nature methods* 2019; 16:1315–22
17. Yang KK, Fusi N, and Lu AX. Convolutions are competitive with transformers for protein sequence pretraining. *Cell systems* 2024; 15:286–94
18. Bommasani R, Hudson DA, Adeli E, Altman R, Arora S, Arx S von, Bernstein MS, Bohg J, Bosselut A, Brunskill E, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* 2021
19. Bepler T and Berger B. Learning the protein language: Evolution, structure, and function. *CELL SYSTEMS* 2021 Jun; 12:654+. DOI: 10.1016/j.cels.2021.05.017
20. Ofer D, Brandes N, and Linial M. The language of proteins: NLP, machine learning & protein sequences. *COMPUTATIONAL AND STRUCTURAL BIOTECHNOLOGY JOURNAL* 2021; 19:1750–8. DOI: 10.1016/j.csbj.2021.03.022
21. Meier J, Rao R, Verkuil R, Liu J, Sercu T, and Rives A. Language models enable zero-shot prediction of the effects of mutations on protein function. *bioRxiv* 2021. DOI: 10.1101/2021.07.09.450648. eprint: <https://www.biorxiv.org/content/early/2021/11/17/2021.07.09.450648.full.pdf>. Available from: <https://www.biorxiv.org/content/early/2021/11/17/2021.07.09.450648>
22. Tzavella K, Diaz A, Olsen C, and Vranken W. Combining evolution and protein language models for an interpretable cancer driver mutation prediction with D2Deep. *Briefings in bioinformatics* 2025; 26:bbae664
23. Yu Y, Jiang F, Ma X, Zhang L, Zhong B, Ouyang W, Fan G, Yu H, Hong L, and Li M. Venus-MAXWELL: Efficient Learning of Protein-Mutation Stability Landscapes using Protein Language Models. *Advances in Neural Information Processing Systems* 2026; 38:137282–308
24. Zhang H, Zheng G, Xu Z, Zhao H, Cai S, Huang Y, Zhou Z, and Wei Y. VarDCL: A Multimodal PLM-Enhanced Framework for Missense Variant Effect Prediction via Self-distilled Contrastive Learning. *bioRxiv* 2026 :2026–3

25. Bell RM, Koren Y, and Volinsky C. The bellkor solution to the netflix prize. KorBell team's report to Netflix 2007; 2
26. Mienye ID and Sun Y. A survey of ensemble learning: Concepts, algorithms, applications, and prospects. *Ieee Access* 2022; 10:99129–49
27. Qin Y, Zhang X, Yu S, and Feng G. A survey on representation learning for multi-view data. *Neural Networks* 2025; 181:106842
28. Friedman JH. Greedy function approximation: a gradient boosting machine. *Annals of statistics* 2001 :1189–232
29. Chen T and Guestrin C. Xgboost: A scalable tree boosting system. *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining.* 2016 :785–94
30. Lin Z, Akin H, Rao R, Hie B, Zhu Z, Lu W, Santos Costa A dos, Fazel-Zarandi M, Sercu T, Candido S, et al. Language models of protein sequences at the scale of evolution enable accurate structure prediction. *BioRxiv* 2022; 2022:500902
31. EvolutionaryScale Team. ESM Cambrian: Revealing the mysteries of proteins with unsupervised learning. EvolutionaryScale Blog. Accessed: June 8, 2026. 2024 Dec. Available from: <https://evolutionaryscale.ai/blog/esm-cambrian>
32. Hayes T, Rao R, Akin H, Sofroniew NJ, Oktay D, Lin Z, Verkuil R, Tran VQ, Deaton J, Wiggert M, et al. Simulating 500 million years of evolution with a language model. *Science* 2025; 387:850–8
33. Ruffolo JA, Gray JJ, and Sulam J. Deciphering antibody affinity maturation with language models and weakly supervised learning. 2021. arXiv: 2112.07782 [q-bio.BM]. Available from: <https://arxiv.org/abs/2112.07782>
34. Hou C, Zhao H, and Shen Y. Protein language models trained on biophysical dynamics inform mutation effects. *Proceedings of the National Academy of Sciences* 2026; 123:e2530466123. DOI: 10.1073/pnas.2530466123. eprint: <https://www.pnas.org/doi/pdf/10.1073/pnas.2530466123>. Available from: <https://www.pnas.org/doi/abs/10.1073/pnas.2530466123>
35. Ribeiro MT, Singh S, and Guestrin C. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.* KDD '16. San Francisco, California, USA: Association for Computing Machinery, 2016 :1135–44. DOI: 10.1145/2939672.2939778. Available from: <https://doi.org/10.1145/2939672.2939778>
36. Nijkamp E, Ruffolo J, Weinstein EN, Naik N, and Madani A. ProGen2: Exploring the Boundaries of Protein Language Models. 2022. arXiv: 2206.13517 [cs.LG]. Available from: <https://arxiv.org/abs/2206.13517>
37. Rao R, Bhattacharya N, Thomas N, Duan Y, Chen P, Canny J, Abbeel P, and Song Y. Evaluating protein transfer learning with TAPE. *Advances in neural information processing systems* 2019; 32
38. Dallago C, Mou J, Johnston KE, Wittmann B, Bhattacharya N, Goldman S, Madani A, and Yang KK. FLIP: Benchmark tasks in fitness landscape inference for proteins. *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2).* 2021. Available from: <https://openreview.net/forum?id=p2dMLEwL8tF>
39. Rao RM, Liu J, Verkuil R, Meier J, Canny J, Abbeel P, Sercu T, and Rives A. MSA transformer. *International conference on machine learning.* PMLR. 2021 :8844–56

40. Brandes N, Ofer D, Peleg Y, Rappoport N, and Linial M. ProteinBERT: a universal deep-learning model of protein sequence and function. *Bioinformatics* 2022; 38:2102–10
41. Elnaggar A, Heinzinger M, Dallago C, Rehawi G, Wang Y, Jones L, Gibbs T, Feher T, Angerer C, Steinegger M, et al. ProtTrans: toward understanding the language of life through self-supervised learning. *IEEE transactions on pattern analysis and machine intelligence* 2021; 44:7112–27
42. Krizhevsky A, Sutskever I, and Hinton GE. ImageNet classification with deep convolutional neural networks. *Communications of the ACM* 2017; 60:84–90
43. Wolpert DH. Stacked generalization. *Neural networks* 1992; 5:241–59